

SQL + MongoDB — Side by Side

Backend Engineer ke liye Database Course • PDF 1 of ~5 • Parts 1-4

Is course mein hum SQL aur MongoDB ko **do alag subjects** ki tarah nahi padhenge. Har concept ek baar samjhenge, phir dekhenge ki SQL mein kaise likhte hain aur MongoDB mein kaise — **same business, same data** ke saath. Phir Node.js mein kaise use hota hai.

PDF	Parts	Topics
PDF 1 (ye wala)	1 - 4	Fundamentals, CRUD, Query Operators, Projection
PDF 2	5 - 8	Pagination, JOIN vs \$lookup, GROUP BY vs \$group, Aggregation
PDF 3	9 - 12	Data Modeling, Indexes, EXPLAIN, Transactions
PDF 4	13 - 16	Concurrency, Connection Pool, Replication, Scaling
PDF 5	17 - 19	Node.js integration, Mongoose, Production Scenarios

► IS COURSE MEIN SQL KE DO FLAVOURS

Aapne dono ka difference maanga tha, isliye jahan PostgreSQL aur MySQL alag behave karte hain, wahan dono dikhaunga. Jahan query same hai, wahan ek hi block **SQL (dono)** label ke saath hoga.

Label	Matlab
SQL (dono)	PostgreSQL aur MySQL dono mein same chalega
PostgreSQL / MySQL	Sirf us database ka syntax — difference highlight karne ke liye
MongoDB	mongosh (Mongo shell) syntax
Node.js	pg / mysql2 / mongodb driver, aur Prisma / Sequelize / Mongoose (ORM/ODM)

► NAMING CONVENTION — PEHLE HI CLEAR KAR LETE HAIN

SQL duniya mein column names **snake_case** hote hain (**user_id**), MongoDB/JavaScript duniya mein **camelCase** (**userId**). Hum yahi follow karenge — real companies mein bhi yahi hota hai.

Kyun? (Important gotcha)

- PostgreSQL unquoted names ko lowercase kar deta hai: **userId** → **userid**. Agar table "**userId**" (quotes ke saath) banaya, to har query mein quotes lagane padenge. Isliye SQL mein **snake_case**.
- MySQL column names case-insensitive treat karta hai, lekin table names OS pe depend karte hain (Linux pe case-sensitive). Safe rehne ke liye lowercase **snake_case**.
- ORM (Prisma/Sequelize) dono ko map kar deta hai: DB mein **user_id**, JS code mein **userId**.

Hamara Business: Sonu ki Pipe Shop (E-commerce)

Poore course mein yahi data use hoga. Isko ek baar dhyan se dekh lo — baar baar refer karenge.

users

user_id	name	city	email
1103	Sonu	Mumbai	sonu@shop.in
1104	Priya	Pune	priya@shop.in
1105	Rahul	Delhi	NULL (email nahi diya)

categories

category_id	name
C10	Pipes
C20	Fittings

products

product_id	name	price	category_id	stock	tags (Mongo)
P101	Steel Pipe	500	C10	20	["steel", "industrial"]
P102	PVC Pipe	200	C10	50	["pvc", "home"]
P103	Elbow Joint	80	C20	100	["steel", "home"]
P104	Copper Pipe	900	C10	5	["copper", "industrial"]

orders

order_id	user_id	total	status	created_at
O1001	1103	1500	PAID	2026-09-01
O1002	1104	900	PENDING	2026-09-03
O1003	1103	280	CANCELLED	2026-09-05
O1004	1105	1000	PAID	2026-09-07

order_items

order_id	product_id	quantity	price (us waqt ka)
O1001	P101	3	500
O1002	P104	1	900
O1003	P102	1	200
O1003	P103	1	80
O1004	P101	2	500

payments

payment_id	order_id	amount	method	status
PAY501	O1001	1500	UPI	SUCCESS
PAY502	O1004	1000	CARD	SUCCESS

Note: O1002 PENDING hai (payment nahi hua), O1003 CANCELLED hai. Rahul ka email NULL hai. Ye cases jaan-bujhkar rakhe hain — NULL, missing data, aur status filters sikhane ke liye.

PART 1 — Database Fundamentals

1.1 Database kya hai?

► CONCEPT

Database = ek organized jagah jahan application ka data **permanently** store hota hai, aur jahan se data ko **fast search, update, aur safely share** kiya ja sakta hai.

► WHY?

Node.js server ki memory (variables, arrays) restart hote hi khaali ho jaati hai. Agar Sonu ka order ek JS array mein rakha aur server crash hua — order gaya. Database data ko disk pe rakhta hai, crash ke baad bhi data safe rehta hai, aur 100 requests ek saath aaye to bhi data corrupt nahi hota.

► REAL-WORLD EXAMPLE

Pipe shop ka register socho. Ek register mein customers, ek mein products, ek mein orders. Poori almirah jismein ye saare registers hain = **Database**. Database server (PostgreSQL / MySQL / MongoDB) = wo dukaan jahan kai almirahs ho sakti hain.

► SQL VS MONGODB

SQL (dono)

```
-- Database banana
CREATE DATABASE pipe_shop;

-- Use karna
-- PostgreSQL (psql): \c pipe_shop
-- MySQL:             USE pipe_shop;
```

MongoDB

```
// MongoDB mein database explicitly banana nahi padta.
// "use" karo, aur pehla document insert karte hi DB ban jaata hai.
use pipe_shop
```

► LINE-BY-LINE

- **CREATE DATABASE pipe_shop;** — naya database bana do. SQL mein pehle container banana padta hai.
- **\c pipe_shop** — psql shell ka command hai (SQL nahi). MySQL mein **USE pipe_shop;** hota hai.
- **use pipe_shop** — Mongo shell ko bolo ki is DB pe kaam karna hai. Agar exist nahi karta, to **lazily** (pehle insert pe) banega.

COMMON MISTAKES

- Mongo mein **use pipe_shop** ke baad **show dbs** mein DB nahi dikha — ghabrao mat, abhi tak koi data insert nahi hua isliye.
- Mongo mein typo **use pipe_shpo** — error nahi aayega! Naya DB silently ban jaayega. Node.js connection string mein bhi yahi hota hai.

INTERVIEW QUESTION

Q: Application memory mein data kyun nahi rakhte, database kyun chahiye?

Answer: Memory volatile hai (restart pe gayab), multiple servers memory share nahi kar sakte, aur concurrent access mein consistency nahi milti. Database persistence, concurrent safe access, fast querying (indexes) aur backup/replication deta hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- Database = persistent, shared, queryable storage - Mongo DB lazily create hota hai	Ek DB server pe multiple databases ho sakte hain	Data disk pe kaise file format mein likha jaata hai (storage engine internals)

1.2 Table vs Collection • Row vs Document • Column vs Field

► CONCEPT

Ye teeno ek saath samajhna aasaan hai, kyunki ye ek hierarchy hain:

Output	
SQL	MongoDB
-----	-----
Database	Database
└─ Table (users)	└─ Collection (users)
└─ Row (Sonu)	└─ Document (Sonu)
└─ Column	└─ Field
(city)	(city)

SQL	MongoDB	Simple matlab
Table	Collection	Ek type ki cheezon ka group — saare users, saare orders
Row	Document	Ek single record — Sonu, ya order O1001
Column	Field	Record ki ek property — name, city, price

► WHY? (ASLI DIFFERENCE KYA HAI)

SQL table ek **Excel sheet** jaisa hai — har row mein bilkul same columns, fixed shape. MongoDB collection ek **folder of JSON files** jaisa hai — har document JSON object hai, aur usmein nested objects aur arrays bhi ho sakte hain.

► SQL

SQL (dono)
<pre>CREATE TABLE users (user_id VARCHAR(10) PRIMARY KEY, name VARCHAR(100) NOT NULL, city VARCHAR(50), email VARCHAR(150));</pre>

Ek row (Sonu) table mein aisa dikhta hai:

Output

user_id	name	city	email
1103	Sonu	Mumbai	sonu@shop.in

► MONGODB

MongoDB

```
// Collection bhi lazily banta hai – insert karo aur ban jaayega
db.users.insertOne({
  userId: "1103",
  name: "Sonu",
  city: "Mumbai",
  email: "sonu@shop.in",
  address: { line1: "Andheri East", pincode: "400069" }, // nested object
  phones: ["9800000001", "9800000002"] // array
})
```

Output

```
{
  _id: ObjectId("66f1a2b3c4d5e6f7a8b9c0d1"),
  userId: "1103",
  name: "Sonu",
  city: "Mumbai",
  email: "sonu@shop.in",
  address: { line1: "Andheri East", pincode: "400069" },
  phones: [ "9800000001", "9800000002" ]
}
```

► LINE-BY-LINE

- **CREATE TABLE users (...)** — table ka structure pehle define karo: kaunse columns, kaunse types.
- **VARCHAR(100)** — text, max 100 characters. **NOT NULL** — ye column khaali nahi ho sakta.
- SQL row mein **nested object ya array direct nahi** rakh sakte (classic design mein). Address ke liye alag columns ya alag table banana padega. (PostgreSQL ka **JSONB** aur MySQL ka **JSON** type exception hai — GOOD TO KNOW.)
- **db.users.insertOne({...})** — **db** = current database, **users** = collection, document = JS object jaisa.
- **address: {...}** — nested object (embedded document). **phones: [...]** — array. Mongo mein ye natural hai.
- **_id** — hum ne nahi diya tha, Mongo ne khud add kiya. (1.3 mein detail.)

► OUTPUT KA MATLAB

SQL row flat hai (ek line, har column ek value). Mongo document ek tree hai (object ke andar object/array). Mongo document internally **BSON** (Binary JSON) mein store hota hai — isliye **ObjectId**, **Date**, **Decimal128** jaise extra types milte hain jo plain JSON mein nahi hote.

COMMON MISTAKES

- Sochna ki MongoDB 'schema-less' hai to kuch bhi daal do. Ek document mein `city`, doosre mein `City`, teesre mein `location` — query karte waqt sab toot jaayega.
- SQL table mein comma-separated values daalna (`phones = '9800000001,9800000002'`). Isse search, update, aur indexing sab mushkil. Alag table banao.
- Mongo document ko bahut bada bana dena (unbounded array). Ek document ki max size **16 MB** hai.

INTERVIEW QUESTION

Q: SQL row aur MongoDB document mein main difference kya hai?

Answer: Row flat hota hai aur table ke fixed schema ko follow karta hai. Document ek JSON-like (BSON) object hai jismein nested objects aur arrays ho sakte hain, aur ek hi collection ke documents ka shape alag ho sakta hai. Isliye Mongo mein related data ek document mein embed kar sakte hain, SQL mein usually alag tables banti hain.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Table/Row/Column ↔ Collection/Document/Field mapping • Document nested object + array rakh sakta hai • Mongo max document size 16 MB	• BSON types: ObjectId, Date, Decimal128 • PostgreSQL JSONB / MySQL JSON column	• BSON binary encoding format kaise kaam karta hai

1.3 Primary Key vs _id

► CONCEPT

Primary Key = wo column (ya columns) jo har row ko **uniquely identify** karta hai. Duplicate nahi, NULL nahi. MongoDB mein har document ka `_id` field hota hai — ye hi uska primary key hai. Ye **mandatory** hai.

► WHY?

Do users ka naam 'Sonu' ho sakta hai. Agar order ko 'Sonu' se link kiya to kaunsa Sonu? Isliye har record ka ek unique ID chahiye jo kabhi duplicate na ho.

► SQL — TEEN TAREEKE (POSTGRESQL VS MYSQL DIFFERENCE YAHAN DIKHTA HAI)

PostgreSQL

```
-- (a) Business ID khud do (hamara case: "1103", "01001")
CREATE TABLE users (
  user_id VARCHAR(10) PRIMARY KEY,
  name    VARCHAR(100) NOT NULL
);

-- (b) Auto-increment number — PostgreSQL (modern way)
CREATE TABLE users_auto (
  id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);

-- Purana PostgreSQL style: id SERIAL PRIMARY KEY    (abhi bhi bahut code mein dikhega)

-- (c) UUID — PostgreSQL 13+
CREATE TABLE users_uuid (
  id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL
);
```

MySQL

```
-- (b) Auto-increment — MySQL
CREATE TABLE users_auto (
  id  BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);

-- (c) UUID — MySQL 8 (BINARY(16) mein store karna space bachata hai)
CREATE TABLE users_uuid (
  id  BINARY(16) PRIMARY KEY DEFAULT (UUID_TO_BIN(UUID())),
  name VARCHAR(100) NOT NULL
);
```

► MONGODB

MongoDB

```
// (a) _id mat do → Mongo ObjectId generate karega
db.users.insertOne({ userId: "1103", name: "Sonu" })
// { _id: ObjectId("66f1a2b3c4d5e6f7a8b9c0d1"), userId: "1103", name: "Sonu" }

// (b) Apna _id do — business ID ko hi _id bana do
db.users.insertOne({ _id: "1103", name: "Sonu" })

// Same _id dobara insert karo:
db.users.insertOne({ _id: "1103", name: "Duplicate Sonu" })
// MongoServerError: E11000 duplicate key error collection: pipe_shop.users
//   index: _id_ dup key: { _id: "1103" }
```

► LINE-BY-LINE

- **PRIMARY KEY** — automatically UNIQUE + NOT NULL + ek index ban jaata hai (fast lookup).
- **GENERATED ALWAYS AS IDENTITY** (Postgres) ≈ **AUTO_INCREMENT** (MySQL) — DB khud 1, 2, 3... dega.
- **ObjectId** — 12 bytes: 4 bytes timestamp + 5 bytes random + 3 bytes counter. Isliye roughly time-ordered hota hai aur **ObjectId.getTimestamp()** se creation time nikal sakte ho.
- **E11000** — Mongo ka duplicate key error code. Node.js mein **err.code === 11000** check karke 409 Conflict return karte hain.

► HAMARE COURSE MEIN KYA KARENGE?

Mongo mein hum **_id ko ObjectId rehne denge** aur business ID (`userId: "1103"`) alag field mein rakhenge, uspe **unique index** lagayenge (Part 10). Ye real projects mein common pattern hai. Lekin ye bhi valid hai ki **_id: "1103"** rakh do — ek field aur ek index kam.

ID type	Fayda	Nuksaan	Kab use
Auto-increment (1,2,3)	Chhota, fast, readable	Guessable (/users/5 → /users/6), multiple DB servers pe clash	Internal IDs, single primary DB
UUID	Globally unique, kahin bhi generate	Bada (16 bytes), random UUID v4 index ko fragment karta hai	Distributed systems, public IDs
ObjectId	Unique + roughly time-sorted, app side bhi ban sakta	Mongo-specific, 12 bytes	Mongo default — mostly yahi

COMMON MISTAKES

- Node.js mein `findOne({ _id: "66f1a2b3..." })` — string se search kiya, document nahi mila! ObjectId aur string alag types hain. Driver mein `new ObjectId(id)` karna padta hai (Mongoose ye auto-cast kar deta hai).
- Email ko primary key banana — email change ho sakta hai. Primary key kabhi change nahi honi chahiye.
- Auto-increment IDs public URL mein expose karna — koi bhi /orders/1, /orders/2 try karke data scrape kar sakta hai (authorization zaroor check karo).

INTERVIEW QUESTION

Q: MongoDB ka `_id` aur SQL primary key mein kya similar hai aur kya different?

Answer: Dono record ko uniquely identify karte hain aur dono pe automatic unique index hota hai. Difference: Mongo mein `_id` har document mein compulsory hai aur default ObjectId (time-ordered, client side bhi generate ho sakta hai) hota hai; SQL mein PK hum define karte hain — natural key, auto-increment ya UUID. Mongo mein `_id` ek hi field hai, SQL mein composite primary key (multiple columns) bhi ho sakti hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- PK = unique + not null + indexed <code>_id</code> mandatory, default ObjectId - String vs ObjectId mismatch bug - E11000 duplicate key error	- ObjectId ka structure (timestamp andar hai) - Auto-increment vs UUID tradeoff - IDENTITY vs SERIAL vs AUTO_INCREMENT	- UUID v7 vs v4 internals - Clustered index internals

1.4 Foreign Key vs Reference

► CONCEPT

Foreign Key = ek table ka column jo doosri table ke primary key ko point karta hai. Order O1001 ka `user_id = 1103` — ye 1103 users table mein exist karna **chahiye**. SQL database ye rule **khud enforce** karta hai.

MongoDB mein isko **Reference** kehte hain — order document mein `userId: "1103"` rakh do. Lekin Mongo **check nahi karta** ki 1103 wala user exist karta hai ya nahi. Ye responsibility application (Node.js) ki hai.

► WHY?

Agar kisi ne galti se order `user_id = 9999` ke saath bana diya aur 9999 naam ka user hai hi nahi — to wo 'orphan' order hai. Report mein customer ka naam nahi aayega, invoice nahi banega. FK is galti ko DB level pe rok deta hai.

► SQL

SQL (dono)

```
CREATE TABLE orders (
  order_id  VARCHAR(10) PRIMARY KEY,
  user_id   VARCHAR(10) NOT NULL,
  total     NUMERIC(10,2) NOT NULL,
  status    VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE RESTRICT
);

INSERT INTO orders (order_id, user_id, total, status)
VALUES ('09999', '9999', 100, 'PAID');
```

Output

PostgreSQL:
ERROR: insert or update on table "orders" violates foreign key constraint "fk_orders_user"
DETAIL: Key (user_id)=(9999) is not present in table "users".

MySQL:
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails

► MONGODB

MongoDB

```
db.orders.insertOne({ orderId: "09999", userId: "9999", total: 100, status: "PAID" })
// { acknowledged: true, insertedId: ObjectId("...") }
// ^ Koi error nahi! Mongo ne check hi nahi kiya ki user 9999 hai ya nahi.
```

Node.js

```
// Node.js service mein khud check karna padta hai
async function createOrder(db, { orderId, userId, total }) {
  const user = await db.collection("users").findOne({ userId }, { projection: { _id: 1 } });
  if (!user) {
    throw new NotFoundError(`User ${userId} not found`); // 404
  }
  return db.collection("orders").insertOne({ orderId, userId, total, status: "PENDING" });
}
```

► LINE-BY-LINE

- **NUMERIC(10,2)** — paise ke liye exact decimal. **Kabhi FLOAT mat use karo money ke liye** ($0.1 + 0.2 \neq 0.3$). MySQL mein **DECIMAL(10,2)** (same cheez).
- **CONSTRAINT fk_orders_user** — constraint ko naam diya, error message mein ye naam dikhega — debugging easy.
- **REFERENCES users(user_id)** — ye column users.user_id ko point karta hai.
- **ON DELETE RESTRICT** — agar user ke orders hain to user delete nahi hone dega. Options: **CASCADE** (user delete → orders bhi delete), **SET NULL**, **RESTRICT/NO ACTION**.
- Mongo wale Node.js code mein bhi ek chhota **race condition** hai: check aur insert ke beech user delete ho sakta hai. Part 12–13 mein solve karenge.

COMMON MISTAKES

- Orders/payments pe **ON DELETE CASCADE** — user delete kiya aur saari financial history gayab. Money data pe usually **RESTRICT** + soft delete.
- MySQL mein MyISAM engine pe FK silently ignore hota hai. Hamesha InnoDB (MySQL 5.5+ ka default).
- Mongo mein reference field ka type mismatch: users mein **userId: "1103"** (string), orders mein **userId: 1103** (number). Match nahi hoga, \$lookup khaali aayega.
- FK column pe index bhulna. PostgreSQL FK column pe automatically index **nahi** banata; MySQL (InnoDB) bana deta hai.

INTERVIEW QUESTION

Q: MongoDB mein foreign key constraint nahi hai, to data integrity kaise maintain karoge?

Answer: Application layer pe: insert se pehle referenced document check karna, Mongoose schema validation, zaroorat ho to multi-document transaction, aur deletes ke time related data handle karna (soft delete ya cleanup job). Saath hi data model aise design karna ki jo cheezein saath change hoti hain wo ek hi document mein embedded hon — single-document operations Mongo mein atomic hote hain.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• FK = DB-enforced relationship • Mongo reference = sirf value, koi enforcement nahi • ON DELETE CASCADE / RESTRICT ka matlab • Money ke liye NUMERIC/DECIMAL, float nahi	• Postgres FK pe index manually banana • MySQL InnoDB vs MyISAM ka FK wala point • Mongo \$jsonSchema validator	• Deferrable constraints • FK check ka internal locking behaviour

1.5 Schema

► CONCEPT

Schema = data ka shape/blueprint — kaunse fields, kaunsa type, kya required hai.

- **SQL: schema-on-write** — likhte waqt hi DB check karta hai. Galat type ya missing required column → reject.
- **MongoDB: flexible schema** — DB by default kuch check nahi karta. Schema **exist karta hai**, bas enforce application (Mongoose) ya optional validator karta hai. Isko 'schema-less' kehna galat hai — 'flexible schema' sahi term hai.

► WHY?

Strict schema = data hamesha clean, lekin naya column add karna = migration. Flexible schema = naya field turant add, lekin purane documents mein wo field nahi hoga — code ko handle karna padega.

► SQL — SCHEMA CHANGE (MIGRATION)

SQL (dono)

```
ALTER TABLE products ADD COLUMN stock INT NOT NULL DEFAULT 0;

INSERT INTO products (product_id, name, price, category_id)
VALUES ('P105', 'Brass Valve', 'abc', 'C20');
-- ERROR: invalid input syntax for type numeric: "abc"      (PostgreSQL)
```

Note: MySQL ka behaviour **sql_mode** pe depend karta hai. MySQL 5.7+ mein **STRICT_TRANS_TABLES** default on hai — error aayega. Strict mode off ho to MySQL 'abc' ko silently 0 bana deta hai (warning ke saath) — dangerous!

► MONGODB — DEFAULT BEHAVIOUR

MongoDB

```
db.products.insertOne({ productId: "P105", name: "Brass Valve", price: "abc" })
// Chal gaya! price string hai, koi error nahi.
```

► MONGODB — DB LEVEL VALIDATION (OPTIONAL)

MongoDB

```
db.createCollection("products", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["productId", "name", "price", "categoryId"],
      properties: {
        productId: { bsonType: "string" },
        name:      { bsonType: "string" },
        price:     { bsonType: ["int", "double", "decimal"], minimum: 0 },
        categoryId: { bsonType: "string" }
      }
    }
  }
})
// Ab price: "abc" → MongoServerError: Document failed validation
```

► NODE.JS — REAL PROJECTS MEIN SCHEMA YAHAN HOTA HAI

Node.js

```
// Mongoose (MongoDB)
const productSchema = new mongoose.Schema({
  productId: { type: String, required: true, unique: true },
  name:      { type: String, required: true, trim: true },
  price:     { type: Number, required: true, min: 0 },
  categoryId: { type: String, required: true },
  stock:     { type: Number, default: 0 },
  tags:      [String]
}, { timestamps: true });
const Product = mongoose.model("Product", productSchema);
```

Node.js

```
// Prisma (schema.prisma) — PostgreSQL ya MySQL, dono ke liye same file
model Product {
  productId String @id @map("product_id") @db.VarChar(10)
  name      String @db.VarChar(100)
  price     Decimal @db.Decimal(10, 2)
  categoryId String @map("category_id")
  stock     Int @default(0)
  category  Category @relation(fields: [categoryId], references: [categoryId])
  @@map("products")
}
// provider = "postgresql" ya "mysql" — datasource block mein badlo, baaki same
```

► LINE-BY-LINE

- **ALTER TABLE ... ADD COLUMN** — SQL mein schema change. Production mein ye **migration file** se hota hai (Prisma Migrate, Knex, Sequelize migrations), manually nahi.
- **\$jsonSchema** — Mongo ka built-in validator. Rarely use hota hai, zyada teams Mongoose pe depend karti hain.
- Mongoose validation sirf **Mongoose ke through** likhe data pe chalta hai. Koi script direct driver se likhe to bypass ho jaayega.
- **@map("product_id")** — Prisma: JS mein **productId**, DB mein **product_id**. Naming wali problem solve.

COMMON MISTAKES

- Mongo 'flexible' hai isliye koi schema hi nahi socha. 6 mahine baad data mein 5 alag shapes — har jagah `if (doc.price ?? doc.cost ?? doc.amount)`.
- Mongoose `updateOne` / `findOneAndUpdate` pe validators by default **nahi chalte**. `{ runValidators: true }` dena padta hai.
- Production mein bade SQL table pe bina soche **ALTER TABLE** — table lock ho sakta hai. (Postgres 11+ mein default ke saath ADD COLUMN fast hai; phir bhi careful.)

INTERVIEW QUESTION

Q: 'MongoDB is schema-less' — ye statement sahi hai?

Answer: Technically galat. MongoDB schema enforce nahi karta by default, lekin application ka data hamesha ek implicit schema follow karta hai. Isliye sahi term 'flexible schema' hai. Enforcement Mongoose ya \$jsonSchema validator se hota hai. Fayda: fast iteration, alag-alag shape ke documents; nuksaan: data quality ki zimmedari app pe.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Schema-on-write (SQL) vs flexible schema (Mongo) • Mongoose schema = practical enforcement • Migrations ka concept • runValidators wala gotcha	• \$jsonSchema validator • MySQL sql_mode strict • Prisma @map	• Online schema change tools (gh-ost, pt-osc)

1.6 Relationships (1:1, 1:N, M:N)

► CONCEPT

Type	Hamare shop mein	SQL mein kaise	MongoDB mein kaise
One-to-One	Order ↔ Payment (ek order ka ek successful payment)	payments.order_id UNIQUE + FK	Payment ko order mein embed, ya alag collection + orderId
One-to-Many	User → Orders (Sonu ke kai orders)	orders.user_id FK (Many wali side pe)	orders mein userId reference (usually)
One-to-Many	Category → Products	products.category_id FK	products mein categoryId
Many-to-Many	Orders ↔ Products (ek order mein kai products, ek product kai orders mein)	Junction table: order_items	Order ke andar items[] array embed

► WHY?

Relationship samajhne se hi decide hota hai ki kitni tables/collections banengi aur data kahan rakha jaayega. System design interview mein pehla sawaal yahi hota hai: 'entities aur unke relationships batao'.

► SQL — MANY-TO-MANY KE LIYE JUNCTION TABLE

SQL (dono)

```
CREATE TABLE order_items (
  order_id  VARCHAR(10) NOT NULL REFERENCES orders(order_id),
  product_id VARCHAR(10) NOT NULL REFERENCES products(product_id),
  quantity  INT NOT NULL CHECK (quantity > 0),
  price     NUMERIC(10,2) NOT NULL,           -- order ke waqt ka price
  PRIMARY KEY (order_id, product_id)         -- composite primary key
);
```

Output

```
order_id | product_id | quantity | price
-----+-----+-----+-----
01001   | P101       | 3        | 500.00
01003   | P102       | 1        | 200.00
01003   | P103       | 1        | 80.00
```

► MONGODB — SAME RELATIONSHIP, EMBEDDED

MongoDB

```
db.orders.insertOne({
  orderId: "01003",
  userId: "1103", // reference (1:N)
  status: "CANCELLED",
  total: 280,
  items: [ // embedded (M:N ko array bana diya)
    { productId: "P102", name: "PVC Pipe", quantity: 1, price: 200 },
    { productId: "P103", name: "Elbow Joint", quantity: 1, price: 80 }
  ],
  createdAt: ISODate("2026-09-05T10:00:00Z")
})
```

► LINE-BY-LINE

- **PRIMARY KEY (order_id, product_id)** — composite key: ek order mein same product do baar row nahi banega (quantity badhao).
- **CHECK (quantity > 0)** — DB level rule. MySQL 8.0.16+ mein CHECK actually enforce hota hai; purane MySQL mein silently ignore hota tha.
- **price** order_items mein kyun? Kal Steel Pipe 600 ka ho gaya to purane order ka bill 500 hi rehna chahiye. Ye **snapshot** hai, duplication nahi.
- Mongo mein **items** array ke andar **name** aur **price** copy kiye — taaki order dikhane ke liye products collection padhni hi na pade.

COMMON MISTAKES

- SQL mein orders table mein **product_ids = 'P101,P102'** — M:N ko ek column mein thoosna. Hamesha junction table.
- Mongo mein user document ke andar **orders: [...]** embed karna — user ke 10,000 orders hue to document 16 MB cross karega. Unbounded growth = reference.
- Order item mein current price ki jagah product se live price padhna — purane bills badal jaayenge.

INTERVIEW QUESTION

Q: E-commerce mein Order aur Product ka relationship kya hai? SQL aur MongoDB mein kaise model karoge?

Answer: Many-to-Many. SQL mein order_items junction table (order_id, product_id, quantity, price snapshot). MongoDB mein order document ke andar items array embed karenge — kyunki items hamesha order ke saath padhe jaate hain, limited hote hain, aur order ke saath hi create hote hain. Product details reference (productId) + snapshot (name, price).

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• 1:1, 1:N, M:N pehchanna • Junction table • FK hamesha 'many' side pe • Price snapshot in order items	• Composite primary key • CHECK constraint (MySQL version caveat)	• Formal ER diagram notation (crow's foot details)

1.7 SQL kab better, MongoDB kab better?

Koi bhi universally better nahi hai. Interview mein 'Mongo fast hai' ya 'SQL purana hai' bola to red flag. Sahi jawab hamesha 'data ke shape aur access pattern pe depend karta hai' + reasons.

Situation	Better fit	Kyun
Bahut saare relationships, complex JOINS, ad-hoc reports (finance, accounting, inventory)	SQL	JOIN, FK, constraints mature hain; normalized data mein ek fact ek jagah
Strict consistency, multi-row transactions har jagah (banking ledger)	SQL	Transactions decades se core feature; Mongo mein bhi hain par design usko avoid karne ka hota hai
Data ka shape har record mein alag (product catalog: pipe ke specs ≠ valve ke specs)	MongoDB	Flexible schema, nested attributes naturally fit
Ek entity hamesha poori padhi jaati hai (order + items + address)	MongoDB	Embed karo → ek read, koi JOIN nahi
Bahut high write volume, horizontal scaling shuru se chahiye (logs, events, IoT)	MongoDB	Built-in sharding; SQL mein bhi possible par zyada kaam
Team ko JS objects hi chahiye, rapid prototyping	MongoDB	Document = JS object, mapping layer kam
Team SQL jaanti hai, reporting/BI tools use honge	SQL	Har BI tool SQL bolta hai

► POSTGRESQL VS MYSQL (EK LINE MEIN)

Dono solid production databases hain. **PostgreSQL**: zyada features (JSONB, RETURNING, rich index types, strict standards) — naye projects mein popular. **MySQL**: simple, bahut widely deployed (WordPress se lekar bade web companies tak), read-heavy workloads mein proven. Backend engineer ke liye 90% syntax same hai.

Real-world truth

- Bahut companies **dono** use karti hain: orders/payments PostgreSQL mein, product catalog/activity logs MongoDB mein. Isko **polyglot persistence** kehte hain.
- PostgreSQL JSONB se document-style data rakh sakta hai, aur MongoDB transactions + \$lookup se relational kaam kar sakta hai. Lines blur ho rahi hain — choice access pattern pe karo.

INTERVIEW QUESTION

Q: Tumhare e-commerce project mein SQL lagaoge ya MongoDB? Justify karo.

Answer: Orders, payments, inventory — strong consistency aur transactions chahiye, relationships clear hain → PostgreSQL. Product catalog jahan har category ke attributes alag hain, ya user activity/events → MongoDB fit hota hai. Agar ek hi DB chunna ho aur team chhoti ho, to PostgreSQL (JSONB flexibility ke saath) safe default hai; agar data mostly self-contained documents hai aur scale horizontal chahiye, to MongoDB. Decision access patterns se.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- Access pattern se decide karna - Koi universally better nahi 3–4 solid reasons dono ke liye	- Polyglot persistence - Postgres JSONB exist karta hai	CAP theorem ki deep theory (Part 15–16 mein practical version aayega)

Part 1 — Memory Table

SQL	MongoDB	Yaad rakhne ka tareeka
Database	Database	Same naam
Table	Collection	Excel sheet ↔ JSON folder
Row	Document	Flat line ↔ JSON tree
Column	Field	Property
PRIMARY KEY	_id	Mongo mein compulsory, default ObjectId
FOREIGN KEY (enforced)	Reference (not enforced)	Mongo mein check = Node.js ka kaam
Junction table (order_items)	Embedded array (items: [])	M:N ka solution
Schema-on-write	Flexible schema (+ Mongoose)	Rules DB mein ↔ rules app mein
ALTER TABLE / migration	Naya field bas likh do	Purane docs mein field missing hoga

PART 2 — CRUD (Create, Read, Update, Delete)

CRUD = backend ka 80% database kaam. Har API endpoint ke peeche inmein se koi operation hota hai:

API	Operation	SQL	MongoDB
POST /users	Create	INSERT	insertOne / insertMany
GET /users, GET /users/:id	Read	SELECT	find / findOne
PATCH /users/:id	Update	UPDATE	updateOne / updateMany
DELETE /users/:id	Delete	DELETE	deleteOne / deleteMany

2.0 Setup — poora sample data load karo (practice ke liye)

Ye script ek baar run kar lo. Aage ke saare examples isi data pe hain. PostgreSQL ke liye

Docker: `docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=pass postgres:16`, MySQL: `docker run -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=pass mysql:8`, Mongo: `docker run -d -p 27017:27017 mongo:7`.

SQL (dono)

```

CREATE TABLE users (
  user_id VARCHAR(10) PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  city    VARCHAR(50),
  email   VARCHAR(150) UNIQUE
);
CREATE TABLE categories (
  category_id VARCHAR(10) PRIMARY KEY,
  name        VARCHAR(50) NOT NULL
);
CREATE TABLE products (
  product_id VARCHAR(10) PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  price       NUMERIC(10,2) NOT NULL CHECK (price >= 0),
  category_id VARCHAR(10) NOT NULL REFERENCES categories(category_id),
  stock       INT NOT NULL DEFAULT 0
);
CREATE TABLE orders (
  order_id   VARCHAR(10) PRIMARY KEY,
  user_id    VARCHAR(10) NOT NULL REFERENCES users(user_id),
  total      NUMERIC(10,2) NOT NULL,
  status     VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE order_items (
  order_id   VARCHAR(10) NOT NULL REFERENCES orders(order_id),
  product_id VARCHAR(10) NOT NULL REFERENCES products(product_id),
  quantity   INT NOT NULL CHECK (quantity > 0),
  price      NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
CREATE TABLE payments (
  payment_id VARCHAR(10) PRIMARY KEY,
  order_id   VARCHAR(10) NOT NULL UNIQUE REFERENCES orders(order_id),
  amount     NUMERIC(10,2) NOT NULL,
  method     VARCHAR(10) NOT NULL,
  status     VARCHAR(20) NOT NULL
);

INSERT INTO users VALUES
  ('1103','Sonu','Mumbai','sonu@shop.in'),
  ('1104','Priya','Pune','priya@shop.in'),
  ('1105','Rahul','Delhi',NULL);
INSERT INTO categories VALUES ('C10','Pipes'), ('C20','Fittings');
INSERT INTO products VALUES
  ('P101','Steel Pipe',500,'C10',20), ('P102','PVC Pipe',200,'C10',50),
  ('P103','Elbow Joint',80,'C20',100), ('P104','Copper Pipe',900,'C10',5);
INSERT INTO orders (order_id,user_id,total,status,created_at) VALUES
  ('01001','1103',1500,'PAID','2026-09-01 10:00:00'),
  ('01002','1104',900,'PENDING','2026-09-03 11:30:00'),
  ('01003','1103',280,'CANCELLED','2026-09-05 09:15:00'),
  ('01004','1105',1000,'PAID','2026-09-07 18:45:00');
INSERT INTO order_items VALUES
  ('01001','P101',3,500), ('01002','P104',1,900),
  ('01003','P102',1,200), ('01003','P103',1,80), ('01004','P101',2,500);
INSERT INTO payments VALUES
  ('PAY501','01001',1500,'UPI','SUCCESS'), ('PAY502','01004',1000,'CARD','SUCCESS');

```

Ye script PostgreSQL aur MySQL 8 dono mein chalegi. (MySQL mein **NUMERIC** automatically **DECIMAL** ban jaata hai.)

```

use pipe_shop

db.users.insertMany([
  { userId: "1103", name: "Sonu", city: "Mumbai", email: "sonu@shop.in",
    address: { line1: "Andheri East", pincode: "400069" } },
  { userId: "1104", name: "Priya", city: "Pune", email: "priya@shop.in",
    address: { line1: "Kothrud", pincode: "411038" } },
  { userId: "1105", name: "Rahul", city: "Delhi" } // email field hi nahi hai
])

db.categories.insertMany([
  { categoryId: "C10", name: "Pipes" },
  { categoryId: "C20", name: "Fittings" }
])

db.products.insertMany([
  { productId: "P101", name: "Steel Pipe", price: 500, categoryId: "C10", stock: 20,
    tags: ["steel", "industrial"] },
  { productId: "P102", name: "PVC Pipe", price: 200, categoryId: "C10", stock: 50,
    tags: ["pvc", "home"] },
  { productId: "P103", name: "Elbow Joint", price: 80, categoryId: "C20", stock: 100,
    tags: ["steel", "home"] },
  { productId: "P104", name: "Copper Pipe", price: 900, categoryId: "C10", stock: 5,
    tags: ["copper", "industrial"] }
])

db.orders.insertMany([
  { orderId: "01001", userId: "1103", total: 1500, status: "PAID",
    shipping: { city: "Mumbai", pincode: "400069" },
    items: [ { productId: "P101", quantity: 3, price: 500 } ],
    createdAt: ISODate("2026-09-01T10:00:00Z") },
  { orderId: "01002", userId: "1104", total: 900, status: "PENDING",
    shipping: { city: "Pune", pincode: "411038" },
    items: [ { productId: "P104", quantity: 1, price: 900 } ],
    createdAt: ISODate("2026-09-03T11:30:00Z") },
  { orderId: "01003", userId: "1103", total: 280, status: "CANCELLED",
    shipping: { city: "Mumbai", pincode: "400069" },
    items: [ { productId: "P102", quantity: 1, price: 200 },
      { productId: "P103", quantity: 1, price: 80 } ],
    createdAt: ISODate("2026-09-05T09:15:00Z") },
  { orderId: "01004", userId: "1105", total: 1000, status: "PAID",
    shipping: { city: "Delhi", pincode: "110001" },
    items: [ { productId: "P101", quantity: 2, price: 500 } ],
    createdAt: ISODate("2026-09-07T18:45:00Z") }
])

db.payments.insertMany([
  { paymentId: "PAY501", orderId: "01001", amount: 1500, method: "UPI", status: "SUCCESS" },
  { paymentId: "PAY502", orderId: "01004", amount: 1000, method: "CARD", status: "SUCCESS" }
])

```

Dhyan do: Mongo mein order_items ki alag collection nahi banayi — items order ke andar embedded hain (1.6 wala design). Aur **shipping** nested object hai — Part 3 mein nested queries isi pe karenge.

2.1 CREATE — INSERT vs insertOne / insertMany

► CONCEPT

Naya record database mein daalna. Naya user signup kare, naya product add ho — INSERT.

► SQL

SQL (dono)

```
INSERT INTO users (user_id, name, city, email)
VALUES ('1106', 'Anita', 'Nagpur', 'anita@shop.in');

-- Multiple rows ek query mein
INSERT INTO users (user_id, name, city) VALUES
  ('1107', 'Vikas', 'Thane'),
  ('1108', 'Meena', 'Surat');
```

PostgreSQL

```
-- Insert karke naya row turant wapas lo (bahut useful API response ke liye)
INSERT INTO users (user_id, name, city)
VALUES ('1109', 'Karan', 'Indore')
RETURNING user_id, name, city;
```

MySQL

```
-- MySQL mein RETURNING nahi hai (MariaDB mein hai).
-- Auto-increment ID chahiye to:
INSERT INTO users_auto (name) VALUES ('Karan');
SELECT LAST_INSERT_ID();
-- Node.js mysql2 mein: result.insertId
```

Output

```
PostgreSQL:  INSERT 0 1
              user_id | name  | city
              +-----+-----+-----
              1109    | Karan | Indore

MySQL:       Query OK, 1 row affected (0.01 sec)
```

► MONGODB

MongoDB

```
db.users.insertOne({ userId: "1106", name: "Anita", city: "Nagpur", email: "anita@shop.in" })

db.users.insertMany([
  { userId: "1107", name: "Vikas", city: "Thane" },
  { userId: "1108", name: "Meena", city: "Surat" }
])
```

Output

```
{ acknowledged: true, insertedId: ObjectId("66f1b0...a1") }

{ acknowledged: true,
  insertedIds: { '0': ObjectId("66f1b0...a2"), '1': ObjectId("66f1b0...a3") } }
```

► LINE-BY-LINE

- **INSERT INTO** `users (user_id, name, city, email)` — kis table mein, kaunse columns bharni hain. **Column list hamesha likho** — table mein naya column aaya to bhi query nahi tootegi.
- **VALUES (...)** — same order mein values. Strings single quotes `'...'` mein. (Double quotes SQL mein identifiers ke liye hote hain, Postgres mein.)
- **RETURNING** — Postgres ka superpower: insert/update/delete ke baad affected rows return. Extra SELECT bach jaata hai.
- **acknowledged: true** — server ne write accept kiya. **insertedId** — naye document ka `_id`.
- **insertMany** by default **ordered** hai: 3rd document fail hua to 4th, 5th insert nahi honge (1st, 2nd ho chuke honge!). `{ ordered: false }` do to baaki continue karenge.

► UPSERT — 'HAI TO UPDATE, NAHI TO INSERT'

PostgreSQL

```
INSERT INTO products (product_id, name, price, category_id, stock)
VALUES ('P101', 'Steel Pipe', 550, 'C10', 20)
ON CONFLICT (product_id)
DO UPDATE SET price = EXCLUDED.price;
```

MySQL

```
INSERT INTO products (product_id, name, price, category_id, stock)
VALUES ('P101', 'Steel Pipe', 550, 'C10', 20) AS new
ON DUPLICATE KEY UPDATE price = new.price;
-- (MySQL 8.0.19+. Purana style: price = VALUES(price))
```

MongoDB

```
db.products.updateOne(
  { product_id: "P101" },
  { $set: { price: 550 }, $setOnInsert: { name: "Steel Pipe", category_id: "C10", stock: 20 } },
  { upsert: true }
)
```

\$setOnInsert — ye fields sirf tab set honge jab naya document bane. Upsert ka use: sync jobs, idempotent APIs, 'last seen' tracking.

COMMON MISTAKES

- **SQL Injection:** Node.js mein ``INSERT ... VALUES ('${name}')` — string concatenation. Hamesha parameterized query (`$1 / ?`). Neeche Node.js section dekho.
- Column list skip karna: **INSERT INTO users VALUES (...)** — kal koi column add hua, saari queries fail.
- Mongo mein **insertMany** fail hua aur samjha kuch insert nahi hua — ordered mode mein failure se pehle wale insert ho chuke hote hain.
- Mongo mein number ko string mein bhejna (`price: "500"`) — `req.body` ka data bina cast kiye daal diya. Baad mein **\$gt** queries galat results dengi.

INTERVIEW QUESTION

Q: Postgres mein INSERT ke baad naya row kaise return karoge? MySQL mein?

Answer: Postgres: **INSERT ... RETURNING ***. MySQL mein RETURNING nahi hai; auto-increment ID **LAST_INSERT_ID()** / mysql2 ka **result.insertId** se milta hai, poora row chahiye to alag SELECT. MongoDB mein **insertOne** **insertedId** deta hai; poora doc chahiye to jo object bheja wahi + `_id`.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• INSERT / insertOne / insertMany • Parameterized queries (injection) • RETURNING (Postgres) • Upsert concept	• ordered vs unordered insertMany • ON CONFLICT vs ON DUPLICATE KEY • \$setOnInsert	• Bulk COPY / LOAD DATA commands • Write concern internals (Part 15 mein basic)

2.2 READ — SELECT vs find

► CONCEPT

Data padhna. Product listing page, order history, admin dashboard — sab READ.

► SQL

SQL (dono)

```
SELECT * FROM users;
```

Output

```

user_id | name  | city  | email
-----+-----+-----+-----
1103    | Sonu  | Mumbai | sonu@shop.in
1104    | Priya | Pune   | priya@shop.in
1105    | Rahul | Delhi  | NULL
(3 rows)
```

► MONGODB

MongoDB

```
db.users.find()
db.users.find({}) // same cheez — empty filter = sab documents
```

Output

```
[
  { _id: ObjectId("..."), userId: "1103", name: "Sonu", city: "Mumbai",
    email: "sonu@shop.in", address: { line1: "Andheri East", pincode: "400069" } },
  { _id: ObjectId("..."), userId: "1104", name: "Priya", city: "Pune",
    email: "priya@shop.in", address: { line1: "Kothrud", pincode: "411038" } },
  { _id: ObjectId("..."), userId: "1105", name: "Rahul", city: "Delhi" }
]
```

► LINE-BY-LINE

- **SELECT *** — saare columns. **FROM users** — is table se. Koi WHERE nahi = saari rows.
- **find()** — ek **cursor** return karta hai, array nahi. mongosh default mein pehle 20 docs dikhata hai, phir **it** type karke aage.
- Rahul ke SQL row mein email **NULL** hai. Mongo document mein email field **exist hi nahi karta**. Ye farak Part 3 ke **\$exists** mein important hoga.

► CURSOR KYA HAI? (NODE.JS KE LIYE IMPORTANT)

Cursor = ek pointer jo DB se data **batches** mein laata hai. 10 lakh orders ko ek saath memory mein nahi laata. Node.js driver mein **find()** cursor deta hai — **.toArray()** karoge tabhi saara data aayega.

Node.js

```
// Chhota result → toArray theek hai
const users = await db.collection("users").find({}).toArray();

// Bada result (export job) → stream/iterate, memory safe
for await (const order of db.collection("orders").find({ status: "PAID" }))) {
  await writeToCsv(order);
}
```

COMMON MISTAKES

- Production API mein `SELECT * FROM orders / find({}).toArray()` bina LIMIT ke — table 50 lakh rows ki hui aur server ki memory khatam. Hamesha pagination (Part 5).
- `SELECT *` — zaroorat se zyada columns (network + memory waste, aur sensitive columns jaise password_hash leak). Part 4 dekho.
- Node.js mein `const users = db.collection('users').find()` karke `users.length` — cursor pe length nahi hota, `await ... .toArray()` bhool gaye.

INTERVIEW QUESTION

Q: MongoDB find() kya return karta hai? Bade dataset ko Node.js mein kaise process karoge?

Answer: Cursor return karta hai jo data batches mein fetch karta hai. Bade dataset ke liye `toArray()` avoid karke `for await...of` se iterate karo ya stream use karo, taaki memory mein ek batch hi rahe. SQL mein bhi similar: pg-cursor / query stream, ya keyset pagination se chunks mein padhna.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• SELECT / find • find() cursor deta hai • Bina LIMIT ke read = production bug	• for await cursor iteration • SQL streaming (pg-query-stream)	• Cursor batch size tuning

2.3 READ ONE — WHERE vs findOne

► CONCEPT

Ek specific record padhna — `GET /users/1103`.

SQL (dono)

```
SELECT * FROM users WHERE user_id = '1103';
```

MongoDB

```
db.users.findOne({ userId: "1103" })
```

Output

```
SQL:   user_id | name | city   | email
-----+-----+-----+-----
      1103    | Sonu | Mumbai | sonu@shop.in
(1 row)
```

```
Mongo: { _id: ObjectId("..."), userId: "1103", name: "Sonu", city: "Mumbai",
        email: "sonu@shop.in", address: {...} }
```

```
Record nahi mila:   SQL → (0 rows)      Mongo findOne → null
```

► LINE-BY-LINE

- **WHERE user_id = '1103'** — sirf wo rows jahan condition true. SQL mein **ek** = (JS jaisa **===** nahi).
- **findOne(filter)** — pehla matching document ya **null**. Filter ek object hai: **{ field: value }** = equality.
- SQL hamesha **rows ka set** deta hai (0, 1, ya zyada). 'Ek hi' chahiye to Node.js mein **rows[0]**. Mongo **findOne** directly object deta hai.
- Kaunsa document 'pehla' hai? Bina sort ke order guaranteed nahi (SQL mein bhi). Unique field pe query karo ya sort do.

Node.js

```
// Express route - 404 handling
app.get("/users/:id", async (req, res) => {
  const user = await db.collection("users").findOne({ userId: req.params.id });
  if (!user) return res.status(404).json({ error: "User not found" });
  res.json(user);
});
```

COMMON MISTAKES

- Null check bhoolna → **Cannot read properties of null (reading 'name')** → 500 error. Hamesha 404 handle karo.
- **findOne({ _id: req.params.id })** — string vs ObjectId (1.3 yaad karo).
- Unique na hone wale field pe **findOne** (jaise **{ city: "Mumbai" }**) aur assume karna ki sahi record mila.

INTERVIEW QUESTION

Q: **findOne** aur **find().limit(1)** mein kya difference hai?

Answer: Result same hai (max ek doc), lekin **findOne** direct document ya null return karta hai, jabki **find().limit(1)** cursor return karta hai. Practically **findOne** convenient hai single lookup ke liye.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• WHERE / findOne • Not found → null / 0 rows → 404	• Bina sort ke order guaranteed nahi	• —

2.4 UPDATE — UPDATE SET vs updateOne + \$set

► CONCEPT

Existing record badalna. Sonu Pune shift ho gaya, product ka price badla, order PAID hua.

► SQL

SQL (dono)

```
UPDATE users
SET city = 'Pune'
WHERE user_id = '1103';

-- Multiple columns + calculation
UPDATE products
SET price = price * 1.10,      -- 10% price badhao
    stock = stock - 3
WHERE product_id = 'P101';
```

Output

```
PostgreSQL: UPDATE 1
MySQL:      Query OK, 1 row affected
           Rows matched: 1  Changed: 1  Warnings: 0
```

► MONGODB

MongoDB

```
db.users.updateOne(
  { userId: "1103" },           // 1. filter — kisko update karna hai
  { $set: { city: "Pune" } }    // 2. update — kya badalna hai
)

db.products.updateOne(
  { productId: "P101" },
  { $mul: { price: 1.10 }, $inc: { stock: -3 } }
)

// Nested field aur array update
db.users.updateOne({ userId: "1103" }, { $set: { "address.pincode": "411001" } })
db.products.updateOne({ productId: "P101" }, { $push: { tags: "bestseller" } })
db.products.updateOne({ productId: "P101" }, { $pull: { tags: "bestseller" } })
db.users.updateOne({ userId: "1103" }, { $unset: { email: "" } }) // field hata do
```

Output

```
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
```

► LINE-BY-LINE

- **UPDATE users** — kaunsi table. **SET city = 'Pune'** — kya badalna. **WHERE ...** — **kaunsi rows**. WHERE bhoole to **saari rows** update!
- **SET price = price * 1.10** — current value pe calculation, DB ke andar hi. Read-modify-write nahi karna pada.
- **\$set** = SQL SET. **\$inc** = **stock = stock + n**. **\$mul** = multiply. **\$unset** = field remove.
- **"address.pincode"** — **dot notation** se nested field. Quotes zaroori hain.
- **\$push / \$pull** — array mein add/remove. **\$addToSet** — push, lekin duplicate nahi.
- **matchedCount: 1** — filter se kitne mile. **modifiedCount: 1** — kitne actually badle. City pehle se Pune thi to matched 1, modified 0. MySQL ka 'Rows matched / Changed' bhi yahi

batata hai.

► SABSE KHATARNAK MONGO MISTAKE

MongoDB

```
// GALAT — $set bhool gaye (purane shell/driver mein ye poora document REPLACE kar deta tha)
db.users.updateOne({ userId: "1103" }, { city: "Pune" })
// Naye driver/mongosh: MongoInvalidArgumentError: Update document requires atomic operators

// Agar sach mein poora document replace karna hai to explicitly:
db.users.replaceOne({ userId: "1103" }, { userId: "1103", name: "Sonu", city: "Pune" })
```

► UPDATEMANY / MULTIPLE ROWS

SQL (dono)

```
UPDATE orders SET status = 'EXPIRED'
WHERE status = 'PENDING' AND created_at < NOW() - INTERVAL '7 days';    -- PostgreSQL
-- MySQL:      ... AND created_at < NOW() - INTERVAL 7 DAY;
```

MongoDB

```
db.orders.updateMany(
  { status: "PENDING", createdAt: { $lt: new Date(Date.now() - 7*24*60*60*1000) } },
  { $set: { status: "EXPIRED" } }
)
```

Dhyan do: Postgres interval **'7 days'** (string), MySQL **7 DAY** (keyword). Chhota lekin common difference.

► UPDATED DOCUMENT WAPAS CHAHIYE? (API RESPONSE KE LIYE)

SQL (dono)

```
UPDATE users SET city = 'Pune' WHERE user_id = '1103' RETURNING *;    -- PostgreSQL
-- MySQL: UPDATE ... ; phir SELECT ... WHERE user_id = '1103';
```

MongoDB

```
db.users.findOneAndUpdate(
  { userId: "1103" },
  { $set: { city: "Pune" } },
  { returnDocument: "after" }    // default "before" (purana document) hota hai!
)
// Mongoose mein: { new: true }
```

COMMON MISTAKES

- **WHERE bhool gaye** — `UPDATE users SET city = 'Pune';` → saare users Pune mein. Production mein pehle same WHERE ke saath SELECT chala ke dekho. (MySQL client mein **--safe-updates** mode ise rokta hai.)
- Mongo mein **\$set** bhoolna (upar dekha).
- **findOneAndUpdate** se updated doc expect kiya lekin purana mila — `returnDocument: "after" / Mongoose { new: true }`.
- Read → JS mein change → write back (`doc.stock = doc.stock - 1; save()`) — do requests ek saath aayi to ek update lost. **\$inc / SET stock = stock - 1** use karo (Part 13).

INTERVIEW QUESTION

Q: Stock kam karne ke liye pehle document padh ke JS mein minus karke save karna, aur \$inc use karna — kya difference hai?

Answer: Read-modify-write mein race condition hai: do requests same stock (1) padhengi, dono 0 likhengi, lekin 2 items bik gaye. `$inc / SET stock = stock - 1` DB ke andar atomically hota hai — single document/row pe lost update nahi hota. Oversell rokne ke liye filter mein `stock: { $gte: qty } / WHERE stock >= qty` bhi lagao.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• UPDATE ... SET ... WHERE • \$set, \$inc, \$push, \$pull, \$unset • matchedCount vs modifiedCount • WHERE/filter bhoolna = disaster • Atomic update vs read-modify-write	• findOneAndUpdate + returnDocument • RETURNING • \$addToSet, dot notation • replaceOne	• Array positional operators (\$[], \$[elem]) ki deep detail

2.5 DELETE — DELETE vs deleteOne

► CONCEPT

Record hatana. Lekin production mein **hard delete bahut kam** hota hai — zyada tar **soft delete**.

SQL (dono)

```
DELETE FROM users WHERE user_id = '1106';
```

MongoDB

```
db.users.deleteOne({ userId: "1106" })
db.users.deleteMany({ city: "Nagpur" })
```

Output

```
SQL:      DELETE 1                (Postgres)    /    Query OK, 1 row affected (MySQL)
Mongo:    { acknowledged: true, deletedCount: 1 }
```

► FK KA ASAR

PostgreSQL

```
DELETE FROM users WHERE user_id = '1103';
-- ERROR: update or delete on table "users" violates foreign key constraint
--         "orders_user_id_fkey" on table "orders"
```

Sonu ke orders hain, isliye SQL ne delete roka (RESTRICT). Mongo mein `deleteOne({ userId: "1103" })` **chal jaayega** aur orders orphan ho jaayenge.

► SOFT DELETE — PRODUCTION PATTERN

SQL (dono)

```
ALTER TABLE users ADD COLUMN deleted_at TIMESTAMP NULL;  
  
UPDATE users SET deleted_at = CURRENT_TIMESTAMP WHERE user_id = '1103';  
  
-- Har read query mein:  
SELECT * FROM users WHERE deleted_at IS NULL;
```

MongoDB

```
db.users.updateOne({ userId: "1103" }, { $set: { deletedAt: new Date() } })  
  
db.users.find({ deletedAt: null }) // null = field missing ya null, dono match
```

► LINE-BY-LINE

- **DELETE FROM users WHERE ...** — WHERE ke bina **DELETE FROM users;** = saari table khaali.
- **deletedCount** — 0 aaya matlab kuch match nahi hua (already deleted ya galat ID).
- **Soft delete kyun:** audit/legal (orders ka record rakhna zaroori), galti se delete ka recovery, FK tootne se bachao.
- Soft delete ka cost: har query mein **deleted_at IS NULL** yaad rakhna padega (ORM mein 'paranoid' mode / global scope se), unique constraints tricky ho jaate hain (deleted user ka email naya user use nahi kar paayega).

COMMON MISTAKES

- **deleteMany({})** — empty filter = poori collection saaf. Node.js mein **deleteMany({ userId: req.body.userId })** aur **userId** undefined aaya → **{ userId: undefined }** driver ke settings pe depend karke unexpected match kar sakta hai. Input validate karo.
- Orders/payments ka hard delete — finance/tax ke liye history chahiye.
- Mongo mein parent delete kiya, child documents cleanup nahi kiye — orphans.

INTERVIEW QUESTION

Q: Soft delete kya hai aur kab use karoge? Iske downsides?

Answer: Row/doc ko actually delete karne ki jagah **deleted_at** timestamp set karna. Use: audit trail, recovery, related data (orders) intact rakhna. Downsides: har query mein filter lagana, table badhti rehti hai, unique constraints complicated (partial unique index se solve — Part 10), GDPR 'right to erasure' ke liye phir bhi hard delete/anonymize karna padega.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• DELETE / deleteOne / deleteMany • Soft delete pattern • WHERE/filter bhoolna = disaster	• Partial unique index for soft delete (Part 10) • ORM paranoid mode	• TRUNCATE vs DELETE internals • VACUUM (Postgres cleanup)

2.6 Node.js mein CRUD — 5 libraries, same kaam

Aapne dono style maange the — raw drivers aur ORM/ODM. Ek hi repository ke 4 functions: **create**, **findById**, **updateCity**, **remove**.

► A) POSTGRESQL — PG (RAW DRIVER)

Node.js

```
const { Pool } = require("pg");
const pool = new Pool({ connectionString: process.env.PG_URL }); // pool: Part 14

const userRepo = {
  async create({ userId, name, city }) {
    const { rows } = await pool.query(
      "INSERT INTO users (user_id, name, city) VALUES ($1, $2, $3) RETURNING *",
      [userId, name, city] // $1, $2, $3 placeholders → SAFE
    );
    return rows[0];
  },
  async findById(userId) {
    const { rows } = await pool.query("SELECT * FROM users WHERE user_id = $1", [userId]);
    return rows[0] ?? null;
  },
  async updateCity(userId, city) {
    const { rows, rowCount } = await pool.query(
      "UPDATE users SET city = $1 WHERE user_id = $2 RETURNING *", [city, userId];
    );
    return rowCount ? rows[0] : null;
  },
  async remove(userId) {
    const { rowCount } = await pool.query("DELETE FROM users WHERE user_id = $1", [userId]);
    return rowCount > 0;
  },
};
```

► B) MYSQL — MYSQL2 (RAW DRIVER)

Node.js

```
const mysql = require("mysql2/promise");
const pool = mysql.createPool({ uri: process.env.MYSQL_URL });

const userRepo = {
  async create({ userId, name, city }) {
    await pool.execute(
      "INSERT INTO users (user_id, name, city) VALUES (?, ?, ?)", // ? placeholders
      [userId, name, city]
    );
    return this.findById(userId); // RETURNING nahi hai → dobara SELECT
  },
  async findById(userId) {
    const [rows] = await pool.execute("SELECT * FROM users WHERE user_id = ?", [userId]);
    return rows[0] ?? null;
  },
  async updateCity(userId, city) {
    const [result] = await pool.execute(
      "UPDATE users SET city = ? WHERE user_id = ?", [city, userId];
    );
    return result.affectedRows ? this.findById(userId) : null;
  },
  async remove(userId) {
    const [result] = await pool.execute("DELETE FROM users WHERE user_id = ?", [userId]);
    return result.affectedRows > 0;
  },
};
```

Cheez	pg (Postgres)	mysql2 (MySQL)
Placeholder	\$1, \$2	?, ?
Result	{ rows, rowCount }	[rows, fields] array (destructure karo)
Affected rows	rowCount	result.affectedRows
Insert ID	RETURNING id	result.insertId
query vs execute	pool.query hi use hota hai	execute = prepared statement (prefer karo)

► C) MONGODB — OFFICIAL DRIVER

Node.js

```
const { MongoClient } = require("mongodb");
const client = new MongoClient(process.env.MONGO_URL);
await client.connect(); // app start pe ek baar
const users = client.db("pipe_shop").collection("users");

const userRepo = {
  async create({ userId, name, city }) {
    const doc = { userId, name, city, createdAt: new Date() };
    const { insertedId } = await users.insertOne(doc);
    return { _id: insertedId, ...doc };
  },
  findById: (userId) => users.findOne({ userId }),
  updateCity: (userId, city) =>
    users.findOneAndUpdate({ userId }, { $set: { city } }, { returnDocument: "after" }),
  async remove(userId) {
    const { deletedCount } = await users.deleteOne({ userId });
    return deletedCount > 0;
  },
};
// Note: driver v6+ mein findOneAndUpdate direct document return karta hai.
// Purane v4/v5 mein { value: doc } milta tha – upgrade karte waqt common bug.
```

► D) PRISMA (POSTGRESQL / MYSQL — SAME CODE)

Node.js

```
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();

const userRepo = {
  create: (data) => prisma.user.create({ data }),
  findById: (userId) => prisma.user.findUnique({ where: { userId } }),
  updateCity: (userId, city) => prisma.user.update({ where: { userId }, data: { city } }),
  remove: (userId) => prisma.user.delete({ where: { userId } }),
};
// Gotcha: update/delete record na mile to Prisma THROW karta hai (code P2025), null nahi deta.
```

► E) SEQUELIZE (POSTGRESQL / MYSQL)

Node.js

```
const User = sequelize.define("User",
  {
    userId: { type: DataTypes.STRING, primaryKey: true, field: "user_id" },
    name: DataTypes.STRING,
    city: DataTypes.STRING,
  },
  {
    tableName: "users",
    timestamps: false,
    paranoid: false
  });

const userRepo = {
  create: (data) => User.create(data),
  findById: (userId) => User.findByPk(userId),
  async updateCity(userId, city) {
    const [count] = await User.update({ city }, { where: { userId } });
    return count ? User.findByPk(userId) : null;
  },
  remove: async (userId) => (await User.destroy({ where: { userId } })) > 0,
};
```

► F) MONGOOSE (MONGODB)

Node.js

```
const User = mongoose.model("User", new mongoose.Schema({
  userId: { type: String, required: true, unique: true },
  name: { type: String, required: true },
  city: String,
}, { timestamps: true }));

const userRepo = {
  create: (data) => User.create(data),
  findById: (userId) => User.findOne({ userId }).lean(), // lean → plain JS object, fast
  updateCity: (userId, city) =>
    User.findOneAndUpdate({ userId }, { city }, { new: true, runValidators: true }).lean(),
  remove: async (userId) => (await User.deleteOne({ userId })).deletedCount > 0,
};

// Mongoose { city } ko automatically { $set: { city } } bana deta hai.
```

► SQL INJECTION — EK BAAR DEKH LO KYUN PARAMETERIZED ZAROORI HAI

Node.js

```
// GALAT
const id = req.params.id; // attacker bheje: 1103' OR '1'='1
await pool.query(`SELECT * FROM users WHERE user_id = '${id}'`);
// Query ban gayi: SELECT * FROM users WHERE user_id = '1103' OR '1'='1' → SAARE users leak

// SAHI
await pool.query("SELECT * FROM users WHERE user_id = $1", [id]);
// Driver value ko data ki tarah bhejta hai, SQL code ki tarah nahi.
```

COMMON MISTAKES

- Template string se SQL banana (upar).
- Har request pe `new Pool()` / `new MongoClient().connect()` — connection leak. App start pe ek baar (Part 14).
- Prisma update ka P2025 error catch na karna → 404 ki jagah 500.
- Mongoose query mein `.lean()` na lagana jab sirf JSON return karna hai — full Mongoose document objects heavy hote hain.

INTERVIEW QUESTION

Q: Raw driver vs ORM — kab kya use karoge?

Answer: ORM (Prisma/Sequelize/Mongoose): fast development, type safety, migrations, validation, less boilerplate. Raw driver: complex/performance-critical queries, full control, ORM ki generated query slow ho to. Practical answer: ORM default, aur hot paths / reports ke liye raw query (Prisma `$queryRaw`, Mongoose `aggregate`).

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- Parameterized queries - pg vs mysql2 result shape - Mongo driver basic CRUD - Ek ORM achhe se (Prisma ya Sequelize) + Mongoose	- Prisma P2025, Mongoose lean/new - driver v6 findOneAndUpdate change	ORM ke internal query builders

Part 2 — Memory Table

Kaam	SQL	MongoDB	Node (pg / mysql2 / mongo)
Insert one	INSERT INTO t (...) VALUES (...)	insertOne(doc)	rows[0] / insertId / insertedId
Insert many	INSERT ... VALUES (...), (...)	insertMany([...])	—
Read all	SELECT * FROM t	find({})	rows / [rows] / toArray()
Read one	SELECT ... WHERE id = \$1	findOne({ id })	rows[0] ?? null / findOne → null
Update	UPDATE t SET c = v WHERE ...	updateOne(f, { \$set })	rowCount / affectedRows / modifiedCount
Increment	SET stock = stock - 1	{ \$inc: { stock: -1 } }	Atomic — race-safe
Return updated	RETURNING * (PG only)	findOneAndUpdate + after	MySQL: dobara SELECT
Upsert	ON CONFLICT / ON DUPLICATE KEY	{ upsert: true }	—
Delete	DELETE FROM t WHERE ...	deleteOne(f)	rowCount / deletedCount
Soft delete	SET deleted_at = NOW()	\$set: { deletedAt }	Har read mein filter

PART 3 — Query Operators (Filtering)

Part 2 mein sirf `=` use kiya. Real APIs mein filters hote hain: 'price 500 se zyada', 'status PAID ya PENDING', 'naam mein pipe'. SQL mein ye sab **WHERE clause** mein jaata hai, MongoDB mein **filter object** mein.

Mongo filter padhne ka golden rule

- `{ field: value }` → equality.
- `{ field: { $operator: value } }` → comparison. Operator hamesha field ke **andar**.
- `{ $and / $or / $nor: [ ... ] }` → logical. Ye top level pe, array ke saath.
- Ek object mein multiple fields = **implicit AND**.

3.1 Comparison: `=`, `!=`, `>`, `>=`, `<`, `<=`, **BETWEEN**

► WHY?

Product listing mein 'price filter' slider, admin mein 'total 1000 se upar ke orders' — sab comparison.

► SQL

SQL (dono)

```
SELECT product_id, name, price FROM products WHERE price = 500;
SELECT product_id, name, price FROM products WHERE price > 500;
SELECT product_id, name, price FROM products WHERE price >= 500;
SELECT product_id, name, price FROM products WHERE price < 500;
SELECT product_id, name, price FROM products WHERE price <= 200;
SELECT product_id, name, price FROM products WHERE price <> 500;      -- != bhi chalta hai
SELECT product_id, name, price FROM products WHERE price BETWEEN 100 AND 600; -- inclusive
```

► MONGODB

MongoDB

```
db.products.find({ price: 500 })
db.products.find({ price: { $gt: 500 } })
db.products.find({ price: { $gte: 500 } })
db.products.find({ price: { $lt: 500 } })
db.products.find({ price: { $lte: 200 } })
db.products.find({ price: { $ne: 500 } })
db.products.find({ price: { $gte: 100, $lte: 600 } }) // BETWEEN – ek field pe do operators
```

► OUTPUT

Condition	Result (product names)
price = 500	Steel Pipe
price > 500	Copper Pipe (900)
price >= 500	Steel Pipe, Copper Pipe
price < 500	PVC Pipe (200), Elbow Joint (80)
price <= 200	PVC Pipe, Elbow Joint
price != 500	PVC Pipe, Elbow Joint, Copper Pipe

Condition	Result (product names)
$100 \leq \text{price} \leq 600$	Steel Pipe, PVC Pipe

► LINE-BY-LINE

- Operator naam yaad rakhne ka tareeka: **gt** = greater than, **gte** = greater than or equal, **lt** = less than, **lte** = less than or equal, **ne** = not equal.
- BETWEEN 100 AND 600** — dono ends include. Mongo mein `{ $gte: 100, $lte: 600 }` — ek hi object mein dono, ye AND ki tarah kaam karta hai.
- Dates pe bhi same: `created_at >= '2026-09-03' / { createdAt: { $gte: ISODate("2026-09-03") } }`.

COMMON MISTAKES

- Mongo mein type mismatch: `price` number hai aur query `{ price: { $gt: "500" } }` (string) — koi result nahi! Mongo alag types ko compare nahi karta jaise JS karta hai. `req.query` se aayi values **hamesha string** hoti hain — `Number(req.query.minPrice)` karo.
- Date ko string ki tarah store karna ("`05-09-2026`") — sorting aur range query toot jaati hai. Hamesha real Date / TIMESTAMP.
- BETWEEN dates ke saath: **BETWEEN '2026-09-01' AND '2026-09-05'** — 5 Sept ka 00:00 tak hi aayega, 5 Sept 10 baje wala miss. Better: `>= '2026-09-01' AND < '2026-09-06'`.

INTERVIEW QUESTION

Q: Express API mein `?minPrice=500` aaya, Mongo query `{ price: { $gte: req.query.minPrice } }` koi result nahi de rahi. Kyun?

Answer: Query params strings hote hain, `"500"` string hai aur DB mein price number hai. MongoDB type-strict comparison karta hai (BSON type order), to number field string se match nahi karega. Fix: `Number()` / Joi/Zod validation se cast. SQL mein Postgres string ko numeric mein implicitly cast kar deta hai jab parameter type infer hota hai, isliye wahan ye bug kam dikhta hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
<code>\$gt \$gte \$lt \$lte \$ne</code> - Range = ek field pe do operators - Query params string hote hain — cast karo	Date range half-open (<code>>= start AND < end</code>)	BSON type comparison order ki poori list

3.2 IN / NOT IN → \$in / \$nin

SQL (dono)

```
SELECT order_id, status FROM orders WHERE status IN ('PAID', 'PENDING');
SELECT order_id, status FROM orders WHERE status NOT IN ('CANCELLED');
```

MongoDB

```
db.orders.find({ status: { $in: ["PAID", "PENDING"] } }, { orderId: 1, status: 1, _id: 0 })
db.orders.find({ status: { $nin: ["CANCELLED"] } }, { orderId: 1, status: 1, _id: 0 })
```

Output

```
IN ('PAID','PENDING') → 01001 PAID, 01002 PENDING, 01004 PAID
NOT IN ('CANCELLED') → 01001 PAID, 01002 PENDING, 01004 PAID
```

► LINE-BY-LINE

- `IN (...) = status = 'PAID' OR status = 'PENDING'` ka short form. Mongo `$in` array leta hai.
- Node.js mein: `WHERE status = ANY($1)` with `[['PAID','PENDING']]` (pg — array ek parameter). mysql2 mein `IN (?)` with `[['PAID','PENDING']]` — query() array expand karta hai.

COMMON MISTAKES

- **NOT IN + NULL trap (SQL):** `WHERE user_id NOT IN (SELECT user_id FROM x)` — agar subquery mein ek bhi NULL hai to result **khaali**! Kyunki `x <> NULL` unknown hota hai. **NOT EXISTS** use karo.
- `$nin` un documents ko bhi match karta hai jinmein field hai hi nahi. `{ email: { $nin: ["a@b.in"] } }` Rahul (email missing) ko bhi laayega.
- IN list mein 10,000 values bhejna — query slow, alag approach socho (temp table / join).

INTERVIEW QUESTION

Q: SQL mein NOT IN ke saath subquery kab dangerous hai?

Answer: Jab subquery ke result mein NULL ho. SQL three-valued logic (TRUE/FALSE/UNKNOWN) ki wajah se `value NOT IN (1, NULL)` kabhi TRUE nahi hota, poora result empty aata hai. Safe alternative: **NOT EXISTS** ya `LEFT JOIN ... IS NULL`.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• IN / \$in, NOT IN / \$nin • Node mein array parameter kaise bhejna	• NOT IN + NULL trap • \$nin missing fields ko match karta hai	• —

3.3 AND / OR → implicit AND, \$and, \$or

► REQUIREMENT

'Sonu (1103) ke PAID orders' (AND). 'PAID ya PENDING orders' (OR). 'Mumbai ke orders jo PAID hain ya total 1000 se zyada' (dono).

SQL (dono)

```
SELECT order_id FROM orders WHERE user_id = '1103' AND status = 'PAID';

SELECT order_id FROM orders WHERE status = 'PAID' OR status = 'PENDING';

-- AND + OR mix: brackets ZAR00RI
SELECT order_id FROM orders
WHERE user_id = '1103'
  AND (status = 'PAID' OR total > 1000);
```

MongoDB

```
// Implicit AND – sabse common, sabse clean
db.orders.find({ userId: "1103", status: "PAID" })

// Explicit $and (yahi result)
db.orders.find({ $and: [ { userId: "1103" }, { status: "PAID" } ] })

// OR
db.orders.find({ $or: [ { status: "PAID" }, { status: "PENDING" } ] })

// AND + OR mix
db.orders.find({
  userId: "1103",
  $or: [ { status: "PAID" }, { total: { $gt: 1000 } } ]
})
```

Output

```
user 1103 AND PAID          → 01001
PAID OR PENDING             → 01001, 01002, 01004
1103 AND (PAID OR total > 1000) → 01001      (01003 CANCELLED aur 280 hai)
```

► \$AND EXPLICITLY KAB CHAHIYE?

Implicit AND ek object hai, aur JS object mein **same key do baar nahi** ho sakti. Jab same field/operator pe do conditions ho, tab **\$and**:

MongoDB

```
// GALAT – do $or keys, doosri pehli ko overwrite kar degi!
db.orders.find({
  $or: [ { status: "PAID" }, { status: "PENDING" } ],
  $or: [ { userId: "1103" }, { userId: "1104" } ] // sirf ye bachega
})

// SAHI
db.orders.find({
  $and: [
    { $or: [ { status: "PAID" }, { status: "PENDING" } ] },
    { $or: [ { userId: "1103" }, { userId: "1104" } ] }
  ]
})
// → 01001 (1103 PAID), 01002 (1104 PENDING)
```

► LINE-BY-LINE

- SQL mein **AND** ki priority **OR** se zyada hai. **a AND b OR c = (a AND b) OR c**. Mix karo to brackets lagao, hamesha.
- Mongo mein brackets ki zaroorat nahi — structure (nesting) hi grouping hai.
- Same field pe OR ho to **\$in** zyada clean + index-friendly: **{ status: { \$in: ["PAID", "PENDING"] } }**.

COMMON MISTAKES

- SQL: **WHERE user_id = '1103' AND status = 'PAID' OR total > 1000** — brackets bhoole, dusre users ke 1000+ orders bhi aa gaye. **Data leak bug** ban sakta hai.
- Mongo: duplicate key in object (upar). JS silently last value rakhta hai — koi error nahi.

INTERVIEW QUESTION

Q: Mongo mein `{ price: { $gt: 100 }, price: { $lt: 600 } }` sahi hai?

Answer: Nahi. JS object mein duplicate key 'price' — sirf `$lt: 600` bachega. Sahi: `{ price: { $gt: 100, $lt: 600 } }` ya `$and` mein do conditions.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Implicit AND • \$or syntax • SQL mein AND/OR mix pe brackets • Duplicate key trap	• \$or vs \$in (same field) • \$nor	• —

3.4 NULL, IS NULL → null, \$exists

► CONCEPT

Rahul (1105) ne email nahi diya. SQL mein uska email column **NULL** hai (column hai, value nahi). Mongo mein uske document mein **email field hi nahi hai**. Mongo mein teen situations ho sakti hain: field missing, field = null, field = value.

SQL (dono)

```
SELECT user_id, name FROM users WHERE email IS NULL;           -- Rahul
SELECT user_id, name FROM users WHERE email IS NOT NULL;       -- Sonu, Priya

-- GALAT: ye kabhi kuch return nahi karega
SELECT * FROM users WHERE email = NULL;
```

MongoDB

```
db.users.find({ email: null })                                // missing YA null — dono
db.users.find({ email: { $exists: false } })                  // sirf missing
db.users.find({ email: { $exists: true } })                    // field hai (value null bhi ho sakti)
db.users.find({ email: { $ne: null } })                        // field hai AUR null nahi ≈ IS NOT NULL
db.users.find({ email: { $type: "null" } })                    // sirf explicitly null
```

Mongo document	{email: null}	{ \$exists: false }	{ \$exists: true }	{ \$ne: null }
{ name: 'Rahul' } (missing)	✓	✓	✗	✗
{ name: 'X', email: null }	✓	✗	✓	✗
{ name: 'Sonu', email: 'sonu@..' }	✗	✗	✓	✓

► LINE-BY-LINE

- SQL mein NULL = 'pata nahi'. **NULL = NULL** bhi TRUE nahi, UNKNOWN hai. Isliye **IS NULL** special operator.
- Mongo ka `{ email: null }` dono (missing + null) pakadta hai — 'SQL IS NULL' ke sabse kareeb.
- **\$exists** — sirf field ki presence check karta hai, value nahi.

COMMON MISTAKES

- `WHERE email = NULL` — hamesha 0 rows.
- Mongo mein soft-delete ke liye `{ deletedAt: { $exists: false } }` use kiya aur kisi code ne `deletedAt: null` set kar diya — wo user 'deleted' dikhne laga. `{ deletedAt: null }` safer.
- SQL: `COUNT(email)` NULLs ko count nahi karta, `COUNT(*)` karta hai (Part 7).

INTERVIEW QUESTION

Q: Mongo mein `{ field: null }` aur `{ field: { $exists: false } }` mein kya farak hai?

Answer: `{ field: null }` un documents ko match karta hai jahan field missing hai YA value null hai. `$exists: false` sirf missing wale. Flexible schema mein missing field common hai, isliye ye farak important hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• IS NULL (not = NULL) • Mongo: missing vs null • <code>{ field: null }</code> dono pakadta hai	• \$type operator • SQL three-valued logic	• —

3.5 Text search: LIKE / ILIKE → \$regex

► REQUIREMENT

Search box: user ne 'pipe' type kiya → Steel Pipe, PVC Pipe, Copper Pipe dikhne chahiye.

SQL (dono)

```
SELECT name FROM products WHERE name LIKE 'Steel%';      -- 'Steel' se shuru
SELECT name FROM products WHERE name LIKE '%Pipe%';      -- 'Pipe' pe khatam
SELECT name FROM products WHERE name LIKE '%Pipe%';      -- kahin bhi 'Pipe'
SELECT name FROM products WHERE name LIKE '_VC%';        -- _ = exactly ek character
```

PostgreSQL

```
-- Case-insensitive
SELECT name FROM products WHERE name ILIKE '%pipe%';      -- PostgreSQL only
SELECT name FROM products WHERE LOWER(name) LIKE '%pipe%'; -- portable
```

MySQL

```
-- MySQL mein LIKE by default case-INSENSITIVE hai (default collation utf8mb4_0900_ai_ci).
SELECT name FROM products WHERE name LIKE '%pipe%';      -- Steel Pipe, PVC Pipe, Copper Pipe
-- ILIKE MySQL mein exist nahi karta.
```

MongoDB

```
db.products.find({ name: /^Steel/ })                     // 'Steel' se shuru
db.products.find({ name: /Pipe$/ })                       // 'Pipe' pe khatam
db.products.find({ name: /Pipe/ })                       // kahin bhi
db.products.find({ name: { $regex: "pipe", $options: "i" } }) // case-insensitive
```

Output

```
'%pipe%' case-insensitive → Steel Pipe, PVC Pipe, Copper Pipe
'Steel%'                  → Steel Pipe
```

Matlab	SQL LIKE	Mongo regex
Shuru mein	'Steel%'	/^Steel/

Matlab	SQL LIKE	Mongo regex
Aakhir mein	'%Pipe'	/Pipe\$
Kahin bhi	'%Pipe%'	/Pipe/
Ek koi character	—	.
Case-insensitive	PG: ILIKE, MySQL: default	\$options: "i" ya /pipe/i

► PERFORMANCE — YE INTERVIEW MEIN POOCHA JAATA HAI

- **Prefix search** ('Steel%', /^Steel/) index use kar sakta hai — fast.
- **Contains search** ('%pipe%', /pipe/) index use **nahi** kar sakta — poori table scan. Chhote data pe theek, lakhon rows pe slow.
- Case-insensitive regex (/pipe/i) bhi index efficiently use nahi karta (collation index alag topic).
- Real search ke liye: Postgres full-text search / `pg_trgm`, MySQL FULLTEXT, Mongo `$text` index ya Atlas Search, ya Elasticsearch.

► NODE.JS — USER INPUT KO REGEX MEIN DAALNA

Node.js

```
// GALAT: user ne "(a+)+$" type kiya → ReDoS (regex CPU hang), ya "." sab match kare
db.collection("products").find({ name: { $regex: req.query.q, $options: "i" } });

// SAHI: special characters escape karo + length limit
const escapeRegex = (s) => s.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
const q = String(req.query.q ?? "").slice(0, 50);
db.collection("products").find({ name: { $regex: escapeRegex(q), $options: "i" } });

// SQL mein bhi % aur _ user input mein ho to escape karo:
await pool.query("SELECT name FROM products WHERE name ILIKE $1", [`%${q}%`]);
```

COMMON MISTAKES

- Search box ke liye '%term%' bade table pe — har keystroke pe full scan.
- Postgres mein **LIKE** case-sensitive hai — 'pipe' search pe 'Pipe' nahi mila. MySQL se Postgres migrate karte waqt common bug.
- User input direct regex mein — ReDoS / unexpected matches.

INTERVIEW QUESTION

Q: **LIKE** '%abc%' slow kyun hai aur kaise solve karoge?

Answer: Leading wildcard ki wajah se B-tree index use nahi ho sakta (index sorted order pe kaam karta hai, shuruat pata nahi to jump nahi kar sakte), isliye full scan. Solution: prefix search agar kaafi ho; warna full-text index (Postgres tsvector/pg_trgm, MySQL FULLTEXT, Mongo text index/Atlas Search) ya dedicated search engine jaise Elasticsearch.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• LIKE % _ / regex ^ \$ • ILIKE (PG) vs MySQL default case-insensitive • Leading wildcard = no index • User input escape	• Full-text search options exist karte hain • ReDoS	• Collation internals • pg_trgm setup details

3.6 Nested fields (dot notation)

► REQUIREMENT

'Mumbai mein ship hone wale orders' — Mongo mein **shipping.city** nested hai.

MongoDB

```
db.orders.find({ "shipping.city": "Mumbai" }, { orderId: 1, _id: 0 })
// [ { orderId: "01001" }, { orderId: "01003" } ]

db.users.find({ "address.pincode": { $regex: /^400/ } }) // Mumbai pincodes
```

SQL (dono)

```
-- Classic SQL: nested object hota hi nahi — shipping_city ek normal column hoga
SELECT order_id FROM orders WHERE shipping_city = 'Mumbai';
```

PostgreSQL

```
-- Agar JSONB column use kiya ho: orders.shipping JSONB
SELECT order_id FROM orders WHERE shipping->>'city' = 'Mumbai';
```

MySQL

```
-- JSON column: orders.shipping JSON
SELECT order_id FROM orders WHERE shipping->>'$.city' = 'Mumbai';
```

► LINE-BY-LINE

- **"shipping.city"** — dot notation, **quotes compulsory** (JS key mein dot hai).
- Postgres **->>** JSON se text nikalta hai (**->** JSON object deta hai). MySQL mein JSON path **'\$.city'**.

COMMON MISTAKES

- **{ shipping: { city: "Mumbai" } }** — ye **poore object ka exact match** hai! shipping mein pincode bhi hai, to ye match nahi karega. Dot notation use karo.
- Quotes bhoolna: **{ shipping.city: "Mumbai" }** — JS syntax error.

INTERVIEW QUESTION

Q: Mongo mein **{ shipping: { city: 'Mumbai' } }** aur **{ 'shipping.city': 'Mumbai' }** mein kya farak hai?

Answer: Pehla embedded document ka exact match hai — saare fields aur unka order same hona chahiye. Doosra sirf city field compare karta hai. 99% cases mein dot notation chahiye.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Dot notation + quotes • Embedded exact match trap	• Postgres ->> / MySQL JSON path	• JSON indexing in SQL (GIN) — baad mein

3.7 Array queries

► CONCEPT

Mongo mein arrays first-class hain: products.tags, orders.items. SQL mein array ka matlab usually **alag table** + JOIN.

► MONGODB

MongoDB

```
// 1. Array mein ye value hai? (simple equality array ke andar dekhta hai)
db.products.find({ tags: "steel" })
// → Steel Pipe, Elbow Joint

// 2. Inmein se koi bhi ek
db.products.find({ tags: { $in: ["copper", "pvc"] } })
// → PVC Pipe, Copper Pipe

// 3. Saare hone chahiye
db.products.find({ tags: { $all: ["steel", "industrial"] } })
// → Steel Pipe

// 4. Array size
db.orders.find({ items: { $size: 2 } })
// → 01003

// 5. Array of objects – ek field
db.orders.find({ "items.productId": "P101" })
// → 01001, 01004

// 6. Array of objects – SAME element pe multiple conditions
db.orders.find({ items: { $elemMatch: { productId: "P101", quantity: { $gte: 3 } } } })
// → 01001 (sirf wahi item jismein P101 AND qty >= 3)
```

► SQL — SAME REQUIREMENTS

SQL (dono)

```
-- order_items table ke saath (normal SQL design)
SELECT DISTINCT order_id FROM order_items WHERE product_id = 'P101';
-- → 01001, 01004

SELECT order_id FROM order_items WHERE product_id = 'P101' AND quantity >= 3;
-- → 01001 ($elemMatch ka SQL version – ek hi row pe dono condition, naturally)

SELECT order_id FROM order_items GROUP BY order_id HAVING COUNT(*) = 2;
-- → 01003 ($size) – GROUP BY Part 7 mein
```

PostgreSQL

```
-- Postgres native array column: tags TEXT[]
SELECT name FROM products WHERE 'steel' = ANY(tags); -- contains
SELECT name FROM products WHERE tags @> ARRAY['steel','industrial']; -- $all
SELECT name FROM products WHERE tags && ARRAY['copper','pvc']; -- $in (overlap)
```

MySQL

```
-- MySQL mein array type nahi hai – JSON column: tags JSON
SELECT name FROM products WHERE JSON_CONTAINS(tags, '"steel"');
SELECT name FROM products WHERE 'steel' MEMBER OF (tags); -- MySQL 8.0.17+
```

► \$ELEMATCH KYUN ZAROORI HAI — SABSE IMPORTANT TRAP

MongoDB

```
// Maan lo ek order: items: [ { productId: "P101", quantity: 1 }, { productId: "P102", quantity: 5 } ]

db.orders.find({ "items.productId": "P101", "items.quantity": { $gte: 3 } })
// MATCH ho jaayega! Kyunki: koi item P101 hai (pehla) AUR koi item qty>=3 hai (doosra).
// Conditions ALAG-ALAG elements pe satisfy ho gayi.

db.orders.find({ items: { $elemMatch: { productId: "P101", quantity: { $gte: 3 } } } })
// Match NAHI hoga — dono conditions ek hi element pe chahiye.
```

COMMON MISTAKES

- Array of objects pe multiple conditions bina `$elemMatch` — galat results, aur ye bug testing mein pakda nahi jaata.
- `$size` range support nahi karta (`{ $size: { $gt: 2 } }` invalid). Count chahiye to `alag field itemCount` maintain karo ya aggregation.
- `{ tags: ["steel", "industrial"] }` — exact array match (order bhi same). `$all` chahiye tha.

INTERVIEW QUESTION

Q: MongoDB mein `$elemMatch` kab use karte ho?

Answer: Jab array of embedded documents pe multiple conditions ek hi element pe apply karni ho. Bina `$elemMatch` ke har condition array ke kisi bhi element se satisfy ho sakti hai, jo galat matches deta hai. SQL mein ye problem nahi hoti kyunki har item alag row hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• <code>{ tags: 'x' }</code> array ke andar dekhta hai • <code>\$in</code>, <code>\$all</code> • <code>"items.field"</code> • <code>\$elemMatch</code> — must	• <code>\$size</code> limitation • Postgres <code>ANY / @></code> • MySQL <code>JSON_CONTAINS / MEMBER OF</code>	• Multikey index internals (Part 10 mein bas basic)

3.8 Node.js — API query params se filter banana (safely)

Real endpoint: `GET /orders?status=PAID,PENDING&minTotal=500&userId=1103`. Dono DBs ke liye filter dynamically banana.

Node.js

```
// Common validation (Zod) – types yahin fix
const q = z.object({
  status: z.string().optional().transform(s => s?.split(",")),
  minTotal: z.coerce.number().min(0).optional(),
  userId: z.string().max(10).optional(),
}).parse(req.query);

// ----- MongoDB -----
const filter = {};
if (q.userId) filter.userId = q.userId;
if (q.status) filter.status = { $in: q.status };
if (q.minTotal !== undefined) filter.total = { $gte: q.minTotal };
const orders = await db.collection("orders").find(filter).limit(20).toArray();

// ----- PostgreSQL (pg) -----
const where = [], params = [];
if (q.userId) { params.push(q.userId); where.push(`user_id = ${params.length}`); }
if (q.status) { params.push(q.status); where.push(`status = ANY(${params.length})`); }
if (q.minTotal !== undefined) { params.push(q.minTotal); where.push(`total >= ${params.length}`); }
const sql = `SELECT * FROM orders ${where.length ? "WHERE " + where.join(" AND ") : ""} LIMIT 20`;
const { rows } = await pool.query(sql, params);

// ----- Prisma (Postgres/MySQL) -----
await prisma.order.findMany({
  where: { userId: q.userId, status: q.status && { in: q.status },
    total: q.minTotal !== undefined ? { gte: q.minTotal } : undefined },
  take: 20,
}); // undefined fields Prisma ignore kar deta hai

// ----- Mongoose -----
await Order.find(filter).limit(20).lean();
```

► NOSQL INJECTION — MONGO KA SQL INJECTION

Node.js

```
// Login API – GALAT
app.post("/login", async (req, res) => {
  const user = await db.collection("users").findOne({
    email: req.body.email,
    password: req.body.password, // (waise bhi plain password kabhi mat rakho!)
  });
});

// Attacker JSON body bhejta hai:
// { "email": "sonu@shop.in", "password": { "$ne": null } }
// Query ban gayi: password != null → hamesha true → LOGIN BYPASS
```

Fix: input ko type-validate karo (Zod/Joi: `password: z.string()`), ya `String(req.body.password)` cast karo. Mongoose schema ka `sanitizeFilter: true` option ya `express-mongo-sanitize` middleware \$ wali keys hata deta hai. Yahi SQL mein parameterized query ka role hai.

COMMON MISTAKES

- `req.query` / `req.body` ko direct filter mein spread karna: `find(req.query)` — attacker kuch bhi filter bana sakta hai.
- Filter conditions string concatenation se banana SQL mein.

INTERVIEW QUESTION

Q: MongoDB mein SQL injection hota hai?

Answer: Classic SQL string injection nahi, lekin **NoSQL operator injection** hota hai — agar user JSON object bhej sake jismein **\$ne**, **\$gt**, **\$regex** ho aur hum usko bina validate kiye filter mein daal dein. Fix: schema validation (types enforce), sanitization (\$ keys hatao), aur never trust req.body shape.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Dynamic filter building • Input validation + casting • NoSQL injection	• Zod/Joi • sanitizeFilter / express-mongo-sanitize	• —

Part 3 — Memory Table

SQL	MongoDB	Note
=	{ f: v }	Array field pe 'contains' ban jaata hai
<> / !=	\$ne	\$ne missing fields bhi match karta hai
> >= < <=	\$gt \$gte \$lt \$lte	Type match zaroori
BETWEEN a AND b	{ \$gte: a, \$lte: b }	Dono inclusive
IN (...)	\$in: [...]	
NOT IN (...)	\$nin: [...]	SQL NULL trap / Mongo missing field
AND	{ a, b } (implicit) / \$and	Duplicate key → \$and
OR	\$or: [{ }, { }]	SQL mein brackets!
IS NULL	{ f: null }	missing + null dono
IS NOT NULL	{ f: { \$ne: null } }	
(no equivalent)	\$exists	Field presence
LIKE 'abc%'	/^abc/	Index use ho sakta hai
LIKE '%abc%'	/abc/	Full scan
ILIKE (PG)	/abc/i or \$options: 'i'	MySQL LIKE default ci
col (flat) / JSONB ->>	"a.b"	Dot notation
JOIN order_items WHERE ...	\$elemMatch	Same element pe conditions

PART 4 — Projection (Selecting Fields)

4.1 SELECT columns ≈ MongoDB projection

► CONCEPT

Projection = row/document ke **kaunse fields** wapas chahiye. WHERE/filter decide karta hai **kaunsi rows**, projection decide karta hai **kaunse columns**.

Output

```
WHERE / filter → rows ko chhaanta hai (horizontal cut)
SELECT cols / projection → columns ko chhaanta hai (vertical cut)
```

► WHY?

- **Security:** users table mein `password_hash` hai — API response mein kabhi nahi jaana chahiye.
- **Performance:** product listing page ko sirf name + price chahiye, 5 KB ka description nahi. 20 products × 5 KB = bekaar network + JSON parsing.
- **Index-only reads:** agar maange gaye saare fields index mein hain, DB table/document padhe bina jawab de sakta hai (covered query — Part 10).

► SQL

SQL (dono)

```
SELECT name, city
FROM users;

-- Alias (naam badalna) — API response shape match karne ke liye useful
SELECT user_id AS "userId", name, city FROM users;      -- PostgreSQL: camelCase ke liye quotes
SELECT user_id AS userId, name, city FROM users;        -- MySQL: quotes ke bina bhi case preserve
```

Output

```
name | city
-----+-----
Sonu  | Mumbai
Priya | Pune
Rahul | Delhi
```

► MONGODB

MongoDB

```
db.users.find(
  {},                                // 1st argument: filter (WHERE)
  { name: 1, city: 1, _id: 0 }      // 2nd argument: projection (SELECT)
)
```

Output

```
[
  { name: "Sonu", city: "Mumbai" },
  { name: "Priya", city: "Pune" },
  { name: "Rahul", city: "Delhi" }
]
```

► LINE-BY-LINE

- `find(filter, projection)` — pehla argument WHERE, doosra SELECT. `{}` = koi filter nahi.
- `name: 1` — ye field chahiye (**inclusion**). `1` ki jagah `true` bhi likh sakte ho.
- `_id: 0` — `_id` nahi chahiye. **Kyun likhna pada?** Neeche.
- SQL mein **AS** se rename. Mongo `find()` projection mein rename limited hai — full reshape ke liye aggregation `$project` (Part 8).

4.2 `_id` alag behave kyun karta hai?

MongoDB mein `_id` **by default hamesha include** hota hai, chahe projection mein likho ya na likho. Baaki fields ka rule ulta hai — inclusion projection mein jo nahi likha wo nahi aata.

MongoDB

```
db.users.find({}, { name: 1 })
// [ { _id: ObjectId("..."), name: "Sonu" }, ... ]    ← _id khud aa gaya

db.users.find({}, { name: 1, _id: 0 })
// [ { name: "Sonu" }, ... ]
```

Kyun aisa design kiya? `_id` document ki identity hai — aksar app ko update/delete ke liye iski zaroorat padti hai, isliye Mongo safe default rakhta hai. SQL mein aisa kuch nahi — jo SELECT mein likha wahi aayega, primary key bhi nahi.

4.3 Inclusion vs Exclusion projection

MongoDB

```
// Inclusion — sirf ye do (whitelist)
db.users.find({}, { name: 1, city: 1 })

// Exclusion — ye chhod ke sab (blacklist)
db.users.find({}, { address: 0, email: 0 })

// GALAT — mix nahi kar sakte (sirf _id exception hai)
db.users.find({}, { name: 1, email: 0 })
// MongoServerError: Cannot do exclusion on field email in inclusion projection

// SAHI — _id ke saath mix allowed
db.users.find({}, { name: 1, city: 1, _id: 0 })
```

SQL (dono)

```
-- SQL mein 'exclude' ka koi syntax nahi.
-- SELECT * EXCEPT (...) standard SQL / Postgres / MySQL mein nahi hai.
-- Hamesha columns explicitly likho.
SELECT user_id, name, city FROM users;      -- email chhodna hai to bas mat likho
```

► NESTED AUR ARRAY FIELDS KA PROJECTION

MongoDB

```
db.users.find({ userId: "1103" }, { name: 1, "address.pincode": 1, _id: 0 })
// { name: "Sonu", address: { pincode: "400069" } }

db.orders.find({ orderId: "01003" }, { "items.productId": 1, _id: 0 })
// { items: [ { productId: "P102" }, { productId: "P103" } ] }

// Array ka sirf pehla element – $slice
db.orders.find({ orderId: "01003" }, { items: { $slice: 1 }, orderId: 1, _id: 0 })
// { orderId: "01003", items: [ { productId: "P102", quantity: 1, price: 200 } ] }
```

SQL (dono)

```
SELECT order_id, product_id FROM order_items WHERE order_id = '01003';
```

4.4 Computed fields

'Har product ki stock value (price × stock)' — SQL mein SELECT ke andar hi calculation. Mongo `find()` projection mein ye (basic) nahi hota — aggregation chahiye (Part 8). MongoDB 4.4+ mein `find` projection aggregation expressions accept karta hai, lekin readability ke liye aggregation zyada common hai.

SQL (dono)

```
SELECT name, price, stock, price * stock AS stock_value
FROM products;
```

MongoDB

```
db.products.aggregate([
  { $project: { _id: 0, name: 1, price: 1, stock: 1,
    stockValue: { $multiply: ["$price", "$stock"] } } }
])
```

Output

name	price	stock	stock_value
Steel Pipe	500	20	10000
PVC Pipe	200	50	10000
Elbow Joint	80	100	8000
Copper Pipe	900	5	4500

"\$price" — aggregation mein \$ + field name = us field ki value. Filter wale `$gt` se confuse mat karna: operator key hota hai, "\$field" value hota hai.

4.5 Node.js mein projection

Node.js

```
// pg / mysql2 – bas SELECT list
await pool.query("SELECT user_id, name, city FROM users WHERE user_id = $1", [id]);

// MongoDB driver – option ka naam 'projection' hai
await db.collection("users").findOne(
  { userId: id },
  { projection: { name: 1, city: 1, _id: 0 } } // driver mein { projection } wrap zaroori
);

// Prisma
await prisma.user.findUnique({ where: { userId: id }, select: { name: true, city: true } });

// Sequelize
await User.findOne({ id, attributes: ["name", "city"] });

// Mongoose – teen tareeke
await User.findOne({ userId: id }).select("name city -_id").lean();
await User.findOne({ userId: id }, "name city -_id").lean();
await User.findOne({ userId: id }, { name: 1, city: 1, _id: 0 }).lean();
```

► PASSWORD KO KABHI LEAK NA HONE DO — SCHEMA LEVEL

Node.js

```
// Mongoose: select: false → default queries mein kabhi nahi aayega
const userSchema = new mongoose.Schema({
  email: String,
  passwordHash: { type: String, select: false },
});
await User.findOne({ email }); // passwordHash nahi aayega
await User.findOne({ email }).select("+passwordHash"); // login ke waqt explicitly

// Prisma: har jagah select likho, ya ek shared "publicUserSelect" object banao
const publicUser = { userId: true, name: true, city: true };
await prisma.user.findMany({ select: publicUser });
```

COMMON MISTAKES

- Mongo driver mein `findOne(filter, { name: 1 })` — projection ko `{ projection: { ... } }` mein wrap karna bhool gaye. Driver usko options samajhta hai, projection ignore → poora document.
- `SELECT *` → `res.json(rows)` — `password_hash`, internal flags API mein leak.
- Inclusion + exclusion mix (`{ name: 1, email: 0 }`).
- Postgres mein `AS userId` bina quotes → column naam `userid` ban gaya, frontend `userId` dhoondh raha hai.
- Projection se field hataya aur baad mein code mein wahi field use kiya → `undefined`. Mongoose mein ye aur confusing hai kyunki default values dikh sakti hain.

INTERVIEW QUESTION

Q: `SELECT *` production mein kyun avoid karte hain? MongoDB mein equivalent problem kya hai?

Answer: Zaroorat se zyada data network pe aata hai, memory/JSON parsing ka cost, sensitive columns leak ho sakte hain, table mein naya column aaya to API response silently badal jaata hai, aur covering index ka fayda nahi milta. Mongo mein bina projection ke `find()` poora document laata hai — bade embedded arrays ke saath ye aur mehenga hai. Fix: explicit `SELECT list / projection`, aur sensitive fields ko schema level pe hide (Mongoose `select:false`).

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• SELECT cols $\approx$ find(filter, projection) • <code>_id</code> default include, <code>_id: 0</code> • Inclusion/exclusion mix nahi • Driver mein <code>{ projection }</code> wrapper • Sensitive fields kabhi return mat karo	• <code>\$slice</code> • Mongoose <code>select('+field') / select:false</code> • Prisma select object • Postgres alias quotes	• Projection ka query planner pe detailed asar

Part 4 — Memory Table

SQL	MongoDB shell	Node driver / ORM
SELECT name, city	find({}, { name: 1, city: 1 })	{ projection: {...} } / select / attributes
(PK automatically nahi)	<code>_id</code> default aata hai $\rightarrow$ <code>_id: 0</code>	Mongoose <code>'-_id'</code>
Column mat likho	{ field: 0 } exclusion	Mongoose <code>'-field', select:false</code>
col AS alias	<code>\$project: { alias: '\$col' }</code>	Prisma: JS mein map karo
price * stock AS v	<code>\$project + \$multiply</code>	<code>aggregate()</code>
WHERE + SELECT	find(filter, projection)	—

PDF 1 — Final Recap (Parts 1-4)

Ye 15 cheezein bina dekhe bol/likh sako, to PDF 2 ke liye ready ho:

- Table/Row/Column $\leftrightarrow$ Collection/Document/Field mapping aur document mein nested object/array.
- PK vs `_id`; ObjectId vs string bug; E11000.
- FK DB enforce karta hai, Mongo reference nahi — Node.js ko check karna padta hai.
- Schema-on-write vs flexible schema; Mongoose/Prisma schema; migrations.
- 1:1, 1:N, M:N — junction table vs embedded array; order item mein price snapshot.
- SQL vs Mongo — access pattern ke hisaab se 3-4 reasons.
- CRUD dono mein; RETURNING (PG) vs LAST_INSERT_ID (MySQL); upsert.
- `$set` bhoolna, WHERE bhoolna — disaster; `matchedCount` vs `modifiedCount`.
- Atomic `$inc` / SET `x = x - 1` vs read-modify-write.
- Soft delete + uske tradeoffs.
- Parameterized queries (SQL injection) + input validation (NoSQL injection).
- `$gt/$gte/$lt/$lte/$ne/$in/$nin`, implicit AND, `$or`, duplicate key trap.
- IS NULL vs `{f: null}` vs `$exists`.
- LIKE/ILIKE/regex; leading wildcard = full scan.
- Dot notation, `$elemMatch`, projection + `_id: 0`.

Practice task (30 min)

- Setup script dono DBs mein chalao (2.0).
- Ye 5 queries SQL aur Mongo dono mein likho: (1) Mumbai ke saare users ke name, (2) price 100-600 wale products, name ke saath, (3) PAID ya PENDING orders jinka total 900+ hai, (4) jin products ke tags mein 'home' hai, (5) jin users ka email nahi hai.
- Ek Express route banao `GET /products?minPrice=&tag=` — Zod validation + Mongo + pg dono repo ke saath.